Kylin
First-Stage NGCC Digital-Signature Submission Package

Kylin Team

June 30, 2026

Contents

1 Algorithm Description **4**

2 Main Design Differences from SPHINCS$^+$ **5**
2.1 Diverse Layer Heights in the Hypertree . . . 5
2.2 BPORS+FP Overview . . . 5
2.3 The WOTS$^+$ Encoding . . . 6
2.4 Size-Optimal Encoding . . . 6
2.5 Theoretical Performance . . . 9

3 Notation and Supporting Definitions **9**
3.1 Data Types . . . 9
3.2 Functions . . . 9
3.3 Operations . . . 10
3.4 Integer to Byte Conversion (Function `toByte`) . . . 10
3.5 Member Functions (Functions `set, get`) . . . 10
3.6 Cryptographic (Hash) Function Families . . . 11
3.6.1 Tweakable Hash Functions (Functions `T, F, H`) . . . 11
3.6.2 PRF and Message Digest (Functions `PRF, PRF_msg, H_msg`) . . . 11
3.6.3 Hash Function Address Scheme (Structure of `ADRS`) . . . 11

4 CS-WOTS$^+$ One-Time Signatures **13**
4.1 CS-WOTS$^+$ Parameters . . . 13
4.2 CS-WOTS$^+$ Chaining Function (Function `chain`) . . . 13
4.3 CS-WOTS$^+$ Private Key (Function `wots_SKgen`) . . . 13
4.4 CS-WOTS$^+$ Public Key Generation (Function `wots_PKgen`) . . . 15
4.5 CS-WOTS$^+$ Encoding (Function `wots_encode`) . . . 15
4.6 CS-WOTS$^+$ Signature Generation (Function `wots_sign`) . . . 17
4.7 CS-WOTS$^+$ Public-Key Recovery (Function `wots_pkFromSig`) . . . 17

5 The Kylin Hypertree **18**
5.1 (Fixed Input-Length) XMSS . . . 18
5.1.1 XMSS Parameters . . . 18
5.1.2 XMSS Private Key . . . 19
5.1.3 TreeHash (Function `treehash`) . . . 19
5.1.4 XMSS Public Key Generation (Function `xmss_PKgen`) . . . 19
5.1.5 XMSS Signature . . . 19
5.1.6 XMSS Signature Generation (Function `xmss_sign`) . . . 20
5.1.7 XMSS Compute Public Key from Signature (Function `xmss_pkFromSig`) . . . 20
5.2 HT: The Hypertree . . . 21
5.2.1 HT Parameters . . . 21
5.2.2 HT Key Generation (Function `ht_PKgen`) . . . 21
5.2.3 HT Signature . . . 21
5.2.4 HT Signature Generation (Function `ht_sign`) . . . 21
5.2.5 HT Signature Verification (Function `ht_verify`) . . . 24

6 BPORS+FP: Bucket-Thinned Few-Time Signatures **24**
6.1 BPORS+FP Parameters . . . 24
6.2 BPORS+FP Private Key and Public Key . . . 25
6.3 BPORS+FP Signature Generation . . . 25
6.4 BPORS+FP Compute Public Key from Signature . . . 25

7 The Full Kylin Signature Scheme **25**
7.1 Kylin Parameters . . . 28
7.2 Kylin Key Generation (Function `kylin_keygen`) . . . 28
7.3 Kylin Signature . . . 29
7.4 Kylin Signature Generation (Function `kylin_sign`) . . . 29
7.5 Kylin Signature Verification (Function `kylin_verify`) . . . 29

8 Parameter Sets and Hash/XOF Instantiation **29**
8.1 Kylin Parameter Sets . . . 30
8.1.1 Influence of Parameters on Security and Performance . . . 32
8.1.2 Proposed Parameter Sets and Security Levels . . . 32
8.2 Instantiations of Hash Functions . . . 33

9 Design Rationale **33**

10 Security Statements and Analyses **34**
10.1 Security Model and Notation . . . 34
10.2 Security of CS-WOTS-TW . . . 35
10.2.1 The Constant-Sum Antichain Property . . . 35
10.2.2 Single-Instance Game Proof . . . 35
10.2.3 Multi-Instance Security . . . 36
10.3 Hypertree Security of Kylin . . . 37
10.3.1 Proof-Oriented Hypertree Description . . . 37
10.3.2 Reduction Theorem . . . 37
10.4 Concrete Security Against Generic Attacks . . . 38
10.5 Relation to the Tight SPHINCS$^+$ Proof . . . 39
10.6 Other Security Properties . . . 39
10.7 Security of the ICCS Hash/XOF Instantiation . . . 39

11 Performance Evaluation **39**

12 Feature Statements **40**

A Parameter Evaluation Sage Script **43**

B $q = 2^{24}$ Companion Parameter Profiles **46**

1 Algorithm Description

Hash-based signature is one of the most promising candidates for (and perhaps the most conservative approach to) post-quantum digital signatures. An advantage of hash-based signatures is that its (classical as well as quantum) security strength is better understood (and easier to evaluate) than other candidates, by solely relying on the idealized hardness[1] of the cryptographic hash functions.

Stateful signatures. Ralph Merkle proposed a hash-based signature [23] that builds upon Lamport's one-time signature (OTS) [21]. The recent efforts towards improving stateful signatures lead to the eXtended Merkle Signature Scheme (XMSS) [16] and the Leighton-Micali Signature (LMS) [22], standardized by NIST [9] and IETF.

Stateless signatures. In a typical stateful signature, e.g., Merkle's signature scheme (MSS), the signer keeps track of which private key of the OTS has been used to avoid security issues from subsequent reuse. This is however not always possible in many practical scenarios. Goldreich proposed a stateless hash-based signature construction [12, 13] which removes the need for maintaining a local state but results in prohibitively large signatures. Recently, this line of research has renewed interest. By incorporating the hypertree structure, SPHINCS [5] offered a practical instantiation of the Goldreich-style stateless hash-based signature. SPHINCS serves as a basis for subsequent works driven by the NIST PQC standardization process, including Gravity-SPHINCS [3], SPHINCS-Simpira [14], and SPHINCS$^+$ [6]. More recent optimizations and analyses include SPHINCS+C [18], forced-pruning techniques for shortening SPHINCS+-style few-time components [1], improved size-time tradeoffs in hypertree-based settings [19], variable-height/chain-length framework extensions [27], and the SPHINCS-alpha work revisiting constant-sum Winternitz one-time signatures with applications to SPHINCS+ and XMSS [32]. NIST has also published the finalized federal standard for stateless hash-based signatures (SLH-DSA) in FIPS 205 [24].

Table 1: Hash-call and size metrics for the three packaged Kylin families. Key generation, signing, and verification are reported in hash-call proxies; key and signature sizes are in bytes.

Instance	KeyGen	Sign	Verify	Pk	Sk	Sig
Kylin-128	305151	2113169	2062	32	64	6616
Kylin-256	575487	2912474	4312	64	128	26440
Kylin-512	1081855	11566745	16757	128	256	84360

Summary of our Work. We propose Kylin, a stateless hash-based signature scheme, which improves upon the state-of-the-art stateless hash-based signature scheme SPHINCS$^+$, while preserving the core elements that made the original SPHINCS$^+$ a standout project.

The construction combines three algorithmic changes. First, CS-WOTS$^+$ provides a size-optimal constant-sum encoding for Winternitz-type one-time signatures; no DAG-based construction is known to offer better size efficiency for this component [32]. Second, BPORS+FP replaces the bottom few-time layer and uses bucket thinning with a fixed authentication-node budget to reduce the few-time signature payload [28]. Third, Kylin allows diverse hypertree layer heights during parameter selection.

Table 2 gives a compact comparison against the SLH-DSA-SHAKE small parameter sets. For the principal metrics shown there, namely signature size and signing/verification hash-call proxies, Kylin is strictly lower than the corresponding SLH-DSA row at both the 128-bit and 256-bit levels.

Diverse Layer Heights in the Hypertree. Kylin also uses a flexible hypertree profile with per-layer heights $(h_0, \ldots, h_{d-1})$ rather than requiring all d layers to have the same height. This removes the unnecessary divisibility restriction $h \bmod d = 0$ and lets the parameter search choose balanced layer-height vectors whenever the total hypertree height h is not divisible by the number of layers.

The security analysis therefore follows the tight SPHINCS$^+$ proof structure while replacing the WOTS checksum argument by the constant-sum antichain argument and replacing the FORS cover calculation

[1] The design philosophy of symmetric primitives (including hash functions) is that they should only admit generic attacks, otherwise the design is considered to be flawed.

Table 2: SLH-DSA-SHAKE small parameter sets versus Kylin for the principal online and transmission metrics. Signature sizes are in bytes; signing and verification are reported as design-level hash-call proxies. Bold entries are strictly lower than the corresponding SLH-DSA row.

Level	Scheme	Sig	Sign	Verify
128	SLH-DSA-SHAKE-128s	7,856	2,125,312	2,094
128	Kylin-128	**6,616**	**2,113,169**	**2,062**
256	SLH-DSA-SHAKE-256s	29,792	2,918,400	4,427
256	Kylin-256	**26,440**	**2,912,474**	**4,312**

by the BPORS+FP cover term. Based on this adapted analysis, we estimate the concrete generic-security exponents and choose parameters for the packaged profiles.

To facilitate a direct package-level view, Table 1 reports the hash-call proxies and key/signature sizes for `Kylin-128`, `Kylin-256`, and `Kylin-512`. In particular, the table includes key-generation, signing, and verification hash-call metrics under the frozen packaged parameter sets.

We refer to more detailed comparisons for the full spectrum of parameter choices in Section 11. As summarized in Table 5, the packaged Kylin instances provide the current repository-local performance profile for key generation, signing, and verification. For specific scenarios where verification time is critical, the parameters can be re-tuned towards faster verification.

Acknowledgement. We thank the SPHINCS$^+$ team for the original project; parts of our code and documentation derive from their work.

2 Main Design Differences from SPHINCS$^+$

We highlight three changes to the original SPHINCS$^+$ scheme: the CS-WOTS$^+$ one-time signature component, BPORS+FP as the bottom few-time component, and diverse hypertree layer heights. Compared with the original WOTS$^+$scheme, CS-WOTS$^+$ gives a larger message-space-to-signature-size ratio and therefore a smaller one-time signature. BPORS+FP further reduces the bottom few-time payload through bucket thinning and a fixed global authentication-node budget, in the same spirit as recent forced-pruning techniques [28, 1]. As a result, the parameters of the hash-based signature scheme can be re-tuned to reduce the signature size without exacerbating the computation time. In addition, Kylin uses a more flexible hypertree layout with diverse per-layer heights, which lets the packaged parameter sets remain well balanced even when h is not divisible by d.

We explain these optimizations and their technical proofs in this section.

2.1 Diverse Layer Heights in the Hypertree

Earlier parameter profiles often imposed the divisibility condition $h \bmod d = 0$ so that every layer had the same height h/d. Kylin allows per-layer heights $(h_0, \ldots, h_{d-1})$ to be non-uniform as long as $\sum_{i=0}^{d-1} h_i = h$. By default, we assume the layer-height vector is ordered, i.e., $h_0 \le h_1 \le \cdots \le h_{d-1}$.

Let $r = h \bmod d$. Our default balancing rule is:

- choose $d - r$ layers with height $\lfloor h/d \rfloor$,
- choose r layers with height $\lceil h/d \rceil$.

Equivalently, the profile is as balanced as possible and differs by at most one between any two layer heights.

2.2 BPORS+FP Overview

BPORS+FP is the few-time signature component used at the bottom of Kylin's hypertree. It replaces the FORS layer of SPHINCS$^+$ while keeping the same high-level role: the randomized message hash selects a bottom hypertree leaf and produces a digest `md`, BPORS+FP signs this digest, and the bottom XMSS tree signs the recovered BPORS+FP public key rather than the original message.

The component combines two ideas. First, bucket thinning [28] splits each of the K BPORS coordinates into 2^B buckets. For a chosen coordinate and bucket, the signer opens k leaves in a child PORS tree with t leaves. When the same bottom hypertree key is selected repeatedly, the bucket index spreads these uses across many child PORS keys, reducing the probability that a forgery is covered by previously opened child secrets. Second, Kylin uses a fixed authentication-node budget in the style of forced-pruning methods [1]: an explicit counter is tried until the total number of child Octopus nodes and outer bucket-tree nodes fits the global bound $m_{\max}$. Thus a BPORS+FP signature has a fixed-size payload consisting of the counter, Kk revealed child secrets, and $m_{\max}$ authentication-node slots. Detailed parameters and algorithms are given in Section 6.

2.3 The WOTS$^+$ Encoding

The reason that the WOTS$^+$(as well as other Winternitz-type OTS) scheme introduces the checksum is that, without the checksum, an adversary can efficiently forge signatures given a single valid message-signature pair. Given (σ, m), it can forge any m' satisfying $\forall i(m_i \leq m'_i)$ by computing

$$\mathbf{F}^{m'_i}(\mathtt{SK}_i) = \mathbf{F}^{m'_i - m_i}\big(\mathbf{F}^{m_i}(\mathtt{SK}_i)\big).$$

The checksum addresses the issue: an increase in any m_i leads to a decrease in at least one C_i (recall $C = \sum_{i=1}^{l_1}(w-1-m_i)$). Therefore, the adversary cannot forge any (m', C') satisfying both $\forall i(m_i \leq m'_i)$ and $\forall i(C_i \leq C'_i)$ at the same time.

2.4 Size-Optimal Encoding

More formally, the problem of constructing one-time signature reduces to that of building an efficient encoding scheme $\mathsf{Enc} : \mathcal{M} \to \mathcal{C} \subseteq [w]^l$ for some incomparable codeword set $\mathcal{C}$ (see Definition 1). In case of WOTS$^+$, the encoding function Enc simply appends the checksum to the original message. Note that WOTS$^+$ fixes the size of the message to l_1 (i.e., $\mathcal{M} = [w]^{l_1}$) and then constructs as small codewords as possible (minimizing $l - l_1$).

Definition 1 ((In)comparability)**.** *For $a, b \in [w]^l$, we denote by $a \leq b$ if for every $i \in [l]$ we have $a_{i+1} \leq b_{i+1}$. If $a \leq b$ or $b \leq a$, we say that a and b are comparable; otherwise a and b are incomparable. A set $S \subseteq \{a : a \in [w]^l\}$ is said to be incomparable (or called an "antichain" in order theory terminology) if any two elements of S are incomparable.*

We take a slightly different approach to encoding the messages. That is, we first fix the size of the codewords to l, $\mathcal{C} \subseteq [w]^l$, and strive to accommodate as large message space $\mathcal{M}$ as possible. Given that Enc is an injection it is essentially to maximize the size of $\mathcal{C} \subseteq [w]^l$. A natural approach is to encode the codewords such that all elements of every codeword sum to the same value, and therefore the checksum is not explicitly needed. Bos and Chaum [8] studied this approach for the special case of $w = 2$. Vaudenay [30] generalized it to arbitrary w, but he did not provide an encoding algorithm. Perin et al. provided a similar encoding algorithm in [26], but they did not present a size-optimal proof.

Theorem 2.1. *For any $m \in [l(w-1)+1]$, $\mathcal{C}_m \stackrel{\text{def}}{=} \{v \in [w]^l : \sum_{i=1}^{l} v_i = m\}$ is incomparable.*

Proof. Suppose towards contradiction that $\mathcal{C}_m$ (for some fixed $m \in [l(w-1)+1]$) is not incomparable, then there exist distinct $a, b \in \mathcal{C}_m$ s.t. $a \leq b$. There must be an index j such that $a_j < b_j$ (otherwise $a = b$). However, due to equal sum $\sum_i a_i = \sum_i b_i$ we have $\sum_{1 \leq i \leq l \wedge i \neq j}(a_i - b_i) > 0$, and there must exist some $1 \leq k \leq l$ such that $a_k > b_k$, which is a contradiction to $a \leq b$. □ □

Every $\mathcal{C}_m$ gives an encoding scheme but with different size. For $m = 0$ or $m = l(w-1)$, $\mathcal{C}_m$ consists of only a single codeword. We argue that the size of $\mathcal{C}_m$ reaches its maximum in the middle, i.e., when $m = \lfloor \frac{l(w-1)}{2} \rfloor$. One easily verifies that this holds in the binary case (i.e., $w = 2$) where $|\mathcal{C}_m| = \binom{l}{m}$. Perin et al. [26] proved that $|\mathcal{C}_m|$ reaches its maximum when $m = \lfloor \frac{l(w-1)}{2} \rfloor$. We prove a stronger optimality result in Theorem 2.2 that the size of $\mathcal{C}_m$, when $m = \lfloor \frac{l(w-1)}{2} \rfloor$, is not only the largest in all $\mathcal{C}_m$ for $m \in [l(w-1)+1]$ but the largest among all valid sets of codewords.

Theorem 2.2 (Size-optimal encoding)**.** *For every incomparable $\mathcal{C}^* \in P([w]^l)$, it holds that*

$$|\mathcal{C}^*| \leq |\mathcal{C}_{\lfloor \frac{l(w-1)}{2} \rfloor}| \ .$$

We defer its proof to Theorem 2.4, which rephrases Theorem 2.2 in the language of order theory. Prior to that, we discuss how to compute $|\mathcal{C}_m|$ by recursion, and give an explicit construction of encoding messages into $\mathcal{C}_m$ for $m = \lfloor \frac{l(w-1)}{2} \rfloor$. Hereafter, we denote such $\mathcal{C}_m$ with maximal size by $\mathcal{C}$ for brevity.

Counting the size. Now we need to figure out the size of $\mathcal{C}$. As a special case, $|\mathcal{C}| = \binom{l}{\lfloor l/2 \rfloor}$ when $w = 2$. Fix w, let

$$D_{n,m} = |\{v \in [w]^n : \sum_{i=1}^{n} v_i = m\}| \ ,$$

we have their initial values

$$D_{1,m} = 1, \text{ for } m \in \{0, 1, \ldots, w-1\}$$
$$D_{n,m} = 0, \text{ for } 2 \leq n \in \mathbb{Z}, m \in \mathbb{Z}^- \ ,$$

and recurrence relation

$$D_{n,m} = \sum_{i=0}^{w-1} D_{n-1,m-i}, 2 \leq n \in \mathbb{Z}, m \in \{0, 1, \ldots, l(w-1)\} \ . \tag{1}$$

Note when $w = 2$, this method is equivalent to recurrence relation of binomial coefficient, i.e., $\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$.

Let us explain the recurrence relation. To compute $D_{n,m}$, consider the value of its last summand, which could be any value in $\{0, 1, \ldots, w-1\}$. If this value is set to i, the sum of the first $n-1$ elements must be $m - i$. Therefore, we notice that the problem "n elements with sum to m" into those "$n-1$ elements with sum to $m - i$". Thus we can simply count $D_{n,m}$ by accumulating $D_{n-1,m-i}$. Following this method, $D_{l,\lfloor l(w-1)/2 \rfloor}$ gives the size of $\mathcal{C}$.

Related works. $D_{n,m}$ is also the m-th coefficient of $(1 + x + x^2 + \cdots + x^{w-1})^n$. Euler [11] has studied $w = 3, 4, 5$, known as trinomial, quadrinomial and quintinomial coefficients respectively. The generalized form was studied in the literature, e.g., [2, 31, 4]. Actually, we can use an inclusion-exclusion argument to express it as a function of binomial coefficients [15]

$$D_{n,m} = \sum_{s=0}^{\lfloor m/w \rfloor} (-1)^s \binom{n}{s} \binom{m + n - sw - 1}{n - 1} \ .$$

Encoding algorithm. Now we make the construction explicit by giving an efficient encoding algorithm, which maps a message $x \in [|\mathcal{C}|]$ into an element in $\mathcal{C}$. We give the pseudocode of the encoding algorithm in Algorithm 1.

Algorithm 1: Encode:$[|\mathcal{C}|] \to \mathcal{C}$.

```
Function Encode(x)
    Let v be an array of size l;
    m ← ⌊l(w − 1)/2⌋;
    for i ← l ... 1 do
        for j ← 0 ... min(w − 1, m) do
            if x ≥ D_{i−1,s−j} then
                x ← x − D_{i−1,s−j};
            else
                v_{l−i} ← j;
                break;
        m ← m − v_{l−i};
    return v;
```

Let us explain the encoding algorithm. As previously stated, the problem can be divided into several subproblems by considering the value of the first element v_{l-i}. To encode a natural number $x \in [0, D_{i,m})$,

we can simply determine $v_{l-i} = j$ by seeking which j satisfies $x \in [\sum_{k<j} D_{i-1,m-k}, \sum_{k\leq j} D_{i-1,m-k})$. Once the value of v_{l-i} is determined, we proceed to the next terms until all elements are decided.

In order to prove the optimality of the encoding, we need some prerequisites about the order theory. The relationship between one-time signature and order theory has been investigated in [7].

Preliminaries of Order Theory

Definition 2 (Poset)**.** *A poset ($\mathcal{S}$,$\leq$) consists of a set $\mathcal{S}$ together with an antisymmetric, transitive and reflexive binary relation '$\leq$', where are certain pairs $(x, y) \in \mathcal{S}$ are comparable ($x \leq y$ or $y \leq x$).*

Note that a poset does not require all pairs in $\mathcal{S}$ to be comparable, and thus it is also known as a partially ordered set.

Definition 3 ((Anti)chain and decomposition)**.** *A chain (resp., antichain) refers to a subset of a poset, whose every pair of elements is comparable (resp., incomparable). A chain decomposition is a partition of a poset into disjoint chains.*

Theorem 2.3 (Dilworth's theorem [10])**.** *For any finite poset P, the size of P's maximum antichain equals the size of a minimum chain decomposition of P.*

Let $P = ([w]^l, \leq)$ be a finite poset. According to Dilworth's theorem, we can prove that $\mathcal{C}$ is the maximum antichain of P by arguing that (1) $\mathcal{C}$ is an antichain and (2) we can find a chain decomposition whose size equals to $|\mathcal{C}|$.

Theorem 2.4. *$\mathcal{C}$ is the maximum antichain, i.e., $\mathcal{C}$ is size-optimal.*

Proof. We have proved that $\mathcal{C}$ is an antichain in Theorem 2.1. It remains to construct the chain decomposition of size $|\mathcal{C}|$ as follows. Our proof can be viewed as a generalization of the proof of Sperner's theorem [29], which considers the special case for $w = 2$.

Consider poset $S_n = ([w]^n, \leq)$, and we sometimes denote an element of S_n by $(a_1, ..., a_n) \in [w]^n$ or by $c_i \in [w]^n$. We slightly abuse the notation $|(a_1, ..., a_n)| \stackrel{\text{def}}{=} a_1 + ... + a_n$.

We construct the chain decomposition for S_n by induction, where every chain $\{c_1, \ldots, c_m\}$ satisfies the following two properties:

- $|c_{i+1}| = |c_i| + 1, \forall i \in \{1, 2, \ldots, m-1\}$,
- $|c_1| + |c_m| = n \cdot (w-1)$.

The case for $n = 1$ is trivial, i.e., $D_{1,\lfloor (w-1)/2 \rfloor} = 1$, which correspond to the chain of $S_1 = \{(0), (1), \ldots, (w-1)\}$.

Assume that we have a chain decomposition for S_{n-1} satisfying the above two properties, we proceed to the construction of a chain decomposition for S_n. For each chain $c = \{c_1, c_2, \ldots, c_m\}$ (from the chain decomposition of S_{n-1}) satisfying the two properties, we build $k+1$ chains for S_n as follows, where $k = \min(w-1, m-1)$. That is, for every $j \in \{0, ..., k\}$ the j-th chain consists of:

$$(c_1, j) \leq \ldots \leq (c_{m-j}, j) \leq (c_{m-j}, j+1) \leq ... \leq (c_{m-j}, w-1) \ .$$

This yields the $k+1$ chains as shown in Fig. 1 below:

$$\begin{array}{ccccccc} (c_1,0) & \ldots & \ldots & \ldots & (c_m,0) & \ldots & (c_m, w-1) \\ \vdots & \ldots & \ldots & \iddots & \ldots & \ldots & \vdots \\ (c_1,k) & \ldots & (c_{m-k},k) & \ldots & \ldots & \ldots & (c_{m-k}, w-1) \end{array}$$

Figure 1: A demonstration of how a chain from S_{n-1} is expanded into $k+1$ chains for S_n, where every row is an expanded chain. Note that it is not a rectangular matrix (every row has two less elements than the previous).

It is easy to verify that

$$|(c_1, j)| + |(c_{m-j}, w-1)| = |(c_1, 0)| + j + |(c_m, 0)| - j + (w-1) = n(w-1),$$

and every subsequent element increases the sum value of its predecessor by one. Namely, the two properties are preserved for all the constructed chains of S_n.

It remains to argue that all the chains constructed (from the decomposed chains of S_{n-1}) constitute a partition of $[w]^n$. That is, for every $c_i \in S_{n-1}$, each of its augmented elements $(c_i, 0), ..., (c_i, w-1)$ appears in the constructed chains exactly once. Note that every c_i belongs to (and can only belong to) one of the decomposed chains of S_{n-1}, say $c = \{c_1, \ldots, c_m\}$. We discuss the following cases.

Case $m \leq w$: We have $k = m - 1 \leq w - 1$. $[(c_i, 0), \ldots, (c_i, k + 1 - i)]$ appears as the first $(k + 2 - i)$ elements of the i-th column, and then $[(c_i, k+1-i), \ldots, (c_i, w-1)]^T$ as the last $(w+i-k-1)$ elements of the $(k + 2 - i)$-th row in Fig. 1.

Case $m > w$: We have $k = w - 1 < m - 1$. If $1 \leq i \leq m - w + 1$, then $[(c_i, 0), (c_i, w - 1)]$ appears as the i-th column in Fig. 1. Otherwise, $m - w + 1 < i \leq m$. $[(c_i, 0), \ldots, (c_i, m - i)]$ and $[(c_i, m-i), \ldots, (c_i, w-1)]^T$ are the first $m - i + 1$ elements of the i-th column, and the last $(w + i - m)$ elements of the $(m - i + 1)$-th row respectively.

Therefore, we have shown that for every $a \in [w]^{n-1}$, $(a, 0)$, ...,$(a, w - 1)$ appears exactly once in the newly constructed chains, namely, the chains constitutes as a chain decomposition for S_n. Finally, it remains to count the number of chains in the decomposition. The two properties guarantee that every chain contains exactly one element c_{mid} with $|c_{\mathsf{mid}}| = \lfloor l(w-1)/2 \rfloor$ (i.e., $c_{\mathsf{mid}} \in \mathcal{C}$). Thus, the size of chain decomposition $|\mathcal{C}| = D_{l,\lfloor l(w-1)/2 \rfloor}$. This completes the proof that $\mathcal{C}$ is the maximum antichain.

□

2.5 Theoretical Performance

Constant Sum WOTS$^+$ has two advantages over WOTS$^+$.

- Stable computing time. The number of hash function calls is fixed in our construction, in contrast to possibly variable numbers for the signing and verification algorithm of WOTS$^+$. While no timing attacks are identified against the implementations of our construction and WOTS$^+$, stable computing time is always preferable (especially for signing algorithms whose computation involves a private key).
- Reduced signature size and number of hash calls. For instance, a baseline 256-bit-classical parameter choice suggests $w = 16$ and $l = 67$. In our construction, for $w = 16$ we require $l = 66$, which reduces 1.5% in both running time (in terms of the expected number of hash function calls) and size. Together with BPORS+FP and diverse layer heights, this local saving contributes to the principal-metric gains summarized in Table 2.

Although our encoding algorithm costs slightly more than the checksum method, it is less dominant compared to the number of hash function calls used in the signature scheme, which will be confirmed in the performance comparison to SPHINCS$^+$.

3 Notation and Supporting Definitions

In the following we start defining basic mathematical operations on integers and bit strings. From that we work our way to more specific basic methods used later in the specification.

3.1 Data Types

Bytes and byte strings are the fundamental data types. A byte is a sequence of eight bits. The set of bytes is denoted as $\mathbb{B}$. A single byte is denoted as a pair of hexadecimal digits with a leading "`0x`". A byte string is an ordered sequence of zero or more bytes and is denoted as an ordered sequence of hexadecimal characters with a leading "`0x`". For example, `0xe534f0` is a byte string of length 3. An array of byte strings is an ordered, indexed set starting with index 0 in which all byte strings have identical length. We assume big-endian representation for any data types or structures.

3.2 Functions

We define the following functions:

$\lceil x \rceil$ (or $\mathsf{ceil}(x)$) for x a real number returns the smallest integer greater than or equal to x.

$\lfloor x \rfloor$ (or $\mathsf{floor}(x)$) for x a real number returns the largest integer less than or equal to x.

$\log(x)$ for x a non-negative real number returns the logarithm to base 2 of x. In pseudocode this function is written as lg.

$\mathsf{Trunc}_\ell(x)$ truncates the bit-string x to the first ℓ bits.

3.3 Operations

When a and b are integers, mathematical operators are defined as follows:

- $\hat{}$: a^b denotes the result of a raised to the power of b.
- $\cdot$: $a \cdot b$ denotes the product of a and b. This operator is sometimes omitted in the absence of ambiguity, as in usual mathematical notation.
- / : a/b denotes the quotient of a by non-zero b.
- % : $a\%b$ denotes the non-negative remainder of the integer division of a by b.
- + : $a + b$ denotes the sum of a and b.
- − : $a - b$ denotes the difference of a and b.
- ++ : $a++$ denotes incrementing a by 1, i.e., $a = a + 1$.
- << : $a << b$ denotes a logical left shift of a by b positions, for b being non-negative, i.e., $a \cdot 2^b$.
- >> : $a >> b$ denotes a logical right shift of a by b positions, for b being non-negative, i.e., $\mathsf{floor}(a/2^b)$.

The standard order of operations is used when evaluating arithmetic expressions.

Arrays are used in the common way, where the i-th element of an array A is denoted $A[i]$. Byte strings are treated as arrays of bytes where necessary: If X is a byte string, then $X[i]$ denotes its i-th byte, where $X[0]$ is the leftmost, highest order byte.

If A and B are byte strings of equal length, then:

- A AND B denotes the bitwise logical conjunction operation.
- A XOR B (or $A \oplus B$) denotes the bitwise logical exclusive disjunction operation.

When B is a byte and i is an integer, then $B >> i$ denotes the logical right-shift by i positions.

If X is an x-byte string and Y a y-byte string, then $X \| Y$ denotes the concatenation of X and Y,with $X \| Y = X[0]...X[x-1]Y[0]...Y[y-1]$.

3.4 Integer to Byte Conversion (Function `toByte`)

For x and y non-negative integers, we define $Z = \texttt{toByte}(x, y)$ to be the y-byte string containing the binary representation of x in big-endian byte-order.

3.5 Member Functions (Functions `set, get`)

To simplify algorithm descriptions, we assume the existence of member functions. If a complex data structure like a public key `PK` contains a variable `X` then `PK.getX()` returns the value of `X` for this public key. Accordingly, `PK.setX(Y)` sets variable `X` in `PK` to the value held by `Y`. Since camelCase is used for member function names, a value `z` may be referred to as `Z` in the function name, e.g., `getZ`.

3.6 Cryptographic (Hash) Function Families

3.6.1 Tweakable Hash Functions (Functions `T`, `F`, `H`)

Following SPHINCS$^+$, a tweakable hash function takes a public seed `PK.seed` and an address `ADRS` in addition to the message input. This effectively renders hash function calls for each key pair and position in the virtual tree independent. The addressing scheme will be described in Section 3.6.3.

The schemes described in this specification build upon several instantiations of tweakable hash functions of the form

$$\begin{aligned} \mathbf{T}_\ell &: \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{\ell n} \rightarrow \mathbb{B}^n \\ \texttt{md} &\leftarrow \mathbf{T}_\ell(\texttt{PK.seed}, \texttt{ADRS}, M) \ , \end{aligned}$$

mapping and ℓn-byte message M to an n-byte hash value md using an n-byte seed `PK.seed` and a 32-byte address `ADRS`. The function $\mathbf{T}_\ell$ is denoted by `T_l` in pseudocode.

There are two special cases which we rename for consistency with previous descriptions of hash-based signature schemes:

$$\begin{aligned} \mathbf{F} &: \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^n \rightarrow \mathbb{B}^n \\ \mathbf{F} &\stackrel{\text{def}}{=} \mathbf{T}_1 \\ \mathbf{H} &: \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{2n} \rightarrow \mathbb{B}^n \\ \mathbf{H} &\stackrel{\text{def}}{=} \mathbf{T}_2 \end{aligned}$$

3.6.2 PRF and Message Digest (Functions `PRF`, `PRF_msg`, `H_msg`)

Kylin uses a pseudorandom function **PRF** for pseudorandom key generation:

$$\mathbf{PRF} : \mathbb{B}^n \times \mathbb{B}^{32} \rightarrow \mathbb{B}^n \ .$$

In addition, Kylin uses a pseudorandom function $\mathbf{PRF_{msg}}$ to generate randomness for the message compression:

$$\mathbf{PRF_{msg}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^n \ .$$

To compress the message to be signed, Kylin uses an additional keyed hash function $\mathbf{H_{msg}}$ that can process arbitrary length messages:

$$\mathbf{H_{msg}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^m \ .$$

3.6.3 Hash Function Address Scheme (Structure of `ADRS`)

An address `ADRS` is a 32-byte value that follows a defined structure. In addition, it comes with set methods to manipulate the address. We explain the generation of addresses in the following sections where they are used. Essentially, all functions have to keep track of the current context, updating the addresses after each hash call.

There are eight different address type values for the different use cases. One type is used for the hashes in CS-WOTS$^+$ schemes, one is used for compression of the CS-WOTS$^+$ public key, the third is used for hashes within the main Merkle tree construction, two are used for BPORS+FP child-tree hashing and BPORS+FP root compression, one is used for CS-WOTS$^+$ private-key value generation, one is used for BPORS+FP child secret generation, and one is used for BPORS+FP outer bucket-tree hashing. These types largely share a common format. We describe them in more detail, below.

The structure of an address complies with word borders, with a word being 32 bits long in this context. Only the tree address (i.e. the index of a specific subtree in the main tree) is too long to fit a single word: for this, we reserve three words. An address is structured as follows. It always starts with a layer address of one word in the most significant bits, followed by a tree address of three words. These addresses describe the position of a tree within the hypertree. The layer address describes the height of a tree within the hypertree starting from height zero for trees on the bottom layer. The tree address describes the position of a tree within a layer of a multi-tree starting with index zero for the leftmost tree. The next word defines the type of the address. It is set to 0 for a CS-WOTS$^+$ hash address, to 1 for the compression of the CS-WOTS$^+$ public key, to 2 for a hash tree address, to 3 for a BPORS+FP

child-tree address, to 4 for BPORS+FP root compression, to 5 for CS-WOTS$^+$ private-key generation, to 6 for BPORS+FP child secret generation, and to 7 for BPORS+FP outer-tree hashing.

We first describe the CS-WOTS$^+$ address (Fig. 2). In this case, the type word is followed by the key pair address that encodes the index of the CS-WOTS$^+$ key pair within the specified tree. The next word encodes the chain address (i.e. the index of the chain within CS-WOTS$^+$), followed by a word that encodes the address of the hash function call within the chain. Note that for the generation of the secret keys based on `SK.seed` a different type of address is used (see below).

Layer Address	Tree Address		
type = 0	Key Pair Address	Chain Address	Hash Address

Figure 2: CS-WOTS$^+$ hash address.

The second type (Fig. 3) is used to compress the CS-WOTS$^+$ public keys. The type word is set to 1. Similar to the address used within CS-WOTS$^+$, the next word encodes the key pair address. The remaining two words are not needed, and thus remain zero. We zero pad the address to the constant length of 32 bytes.

Layer Address	Tree Address	
type = 1	Key Pair Address	Padding = 0

Figure 3: CS-WOTS$^+$ public key compression address.

The third type (Fig. 4) addresses the hash functions in the main tree. In this case the type word is set to 2, followed by a zero padding of one word. The next word encodes the height of the tree node that is being computed, followed by a word that encodes the index of this node at that height.

Layer Address	Tree Address		
type = 2	Padding = 0	Tree Height	Tree Index

Figure 4: hash tree address.

The next type (Fig. 5) is of a similar format, and is used to describe hash functions in a BPORS+FP child tree. The type word is set to 3. The key pair address identifies the bottom BPORS+FP key pair, identical to the key pair address in the CS-WOTS$^+$ hash addresses. The remaining words encode the BPORS coordinate, bucket, tree height, and tree index as needed by the implementation address setters. To generate child secret values from `SK.seed` a different type of address is used (see below).

The next type (Fig. 6) is used to compress the BPORS+FP outer roots. The type word is set to 4. Like the CS-WOTS$^+$ public key compression address, it contains the address of the BPORS+FP key pair and is padded to the full length.

The type values 5 and 6 are used for secret key value generation in CS-WOTS$^+$ and BPORS+FP. A CS-WOTS$^+$ key generation address (Fig. 7) is the same as a CS-WOTS$^+$ hash address with two differences. First, the type word is set to 5. Second, the hash address word is constantly set to 0. When generating the secret key value for a given chain, the remaining words have to be set the same way as for the CS-WOTS$^+$ hash addresses used for this chain.

Similarly, the BPORS+FP child secret generation type (Fig. 8) is the same as the BPORS+FP child-tree address type, except that the type word is set to 6, and the tree height word is set to 0. As for the CS-WOTS$^+$ key generation address, the remaining words have to be set as for the BPORS+FP child-tree address used when processing the generated value. Type value 7 is used for hashing the complete BPORS+FP outer bucket trees.

All fields within these addresses encode unsigned integers. When describing the generation of addresses we use set methods that take positive integers and set the bits of a field to the binary representation of that integer, in big-endian notation. Throughout this document, we adhere to the convention of assuming that changing the type word of an address (indicated by the use of the `setType()` method) initializes the subsequent three words to zero.

Layer Address	Tree Address		
type = 3	Key Pair Address	Tree Height	Tree Index

Figure 5: BPORS+FP child-tree address.

Layer Address	Tree Address	
type = 4	Key Pair Address	Padding = 0

Figure 6: BPORS+FP roots compression address.

In order to make keeping track of the types easier throughout the pseudo-code in the rest of this document, we refer to them respectively using the constants `WOTS_HASH`, `WOTS_PK`, `TREE`, `BPORS_CHILD_TREE`, `BPORS_ROOTS`, `WOTS_PRF`, `BPORS_CHILD_PRF`, and `BPORS_OUTER`.

4 CS-WOTS$^+$ One-Time Signatures

This section describes the CS-WOTS$^+$ scheme, while a private key can be used to sign any message, each private key MUST NOT be used to sign more than a single message.

4.1 CS-WOTS$^+$ Parameters

CS-WOTS$^+$ uses the parameters n, w and `len`; They are positive integer values. These parameters are summarized as follows:

- n: the security parameter; it is the message length as well as the length of a private key, public key, or signature element in bytes.
- w: the Winternitz parameter;
- `len`: the number of n-byte-string elements in a CS-WOTS$^+$ private key, public key, and signature.

4.2 CS-WOTS$^+$ Chaining Function (Function `chain`)

The chaining function (Algorithm 2) computes an iteration of **F** on an n-byte input using a CS-WOTS$^+$ hash address `ADRS` and a public seed `PK.seed`. The address `ADRS` MUST have the first seven 32-bit words set to encode the address of this chain. In each iteration, the address is updated to encode the current position in the chain before `ADRS` is used to process the input by **F**.

In the following, `ADRS` is a 32-byte CS-WOTS$^+$ hash address as specified in Section 3.6.3 and `PK.seed` is a n-byte string. The chaining function takes as input an n-byte string X, a start index i, a number of steps s, as well as `ADRS` and `PK.seed`. The chaining function returns as output the value obtained by iterating **F** for s times on input X.

4.3 CS-WOTS$^+$ Private Key (Function `wots_SKgen`)

The CS-WOTS$^+$ private key, denoted by `SK`, is a length `len` array of n-byte strings. This private key MUST NOT be used to sign more than one message. This private key is only implicitly used. Therefore, the following is just to support a better understanding of the following algorithms. Each n-byte string in the CS-WOTS$^+$ private key is derived from a secret seed `SK.seed` which is part of the Kylin secret key and a CS-WOTS$^+$ key generation address `skADRS` using **PRF**. The same secret seed is used to generate all secret key values within Kylin. The address used to generate the i-th n-byte string of `SK` MUST encode the position of the i-th hash chain of this CS-WOTS$^+$ instance within the Kylin structure.

The following pseudocode (Algorithm 3) describes an algorithm to generate a CS-WOTS$^+$ private key.

Algorithm 2: `chain` — Chaining function used in CS-WOTS$^+$.

```
# Input: Input string X, start index i, number of steps s, public seed PK.seed,
  ↪ address ADRS
# Output: value of F iterated s times on X
chain(X, i, s, PK.seed, ADRS) {
  if ( s == 0 ) {
    return X;
  }
  if ( (i + s) > (w - 1) ) {
    return NULL;
  }
  byte[n] tmp = chain(X, i, s - 1, PK.seed, ADRS);
  ADRS.setHashAddress(i + s - 1);
  tmp = F(PK.seed, ADRS, tmp);
  return tmp;
}
```

Algorithm 3: `wots_SKgen` — Generating a CS-WOTS$^+$ private key.

```
# Input: secret seed SK.seed, address ADRS
# Output: CS-WOTS+ private key sk
wots_SKgen(SK.seed, ADRS) {
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
    skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk[i] = PRF(SK.seed, skADRS);
  }
  return sk;
}
```

Layer Address	Tree Address		
type = 5	Key Pair Address	Chain Address	Hash Address = 0

Figure 7: CS-WOTS$^+$ key generation address.

Layer Address	Tree Address		
type = 6	Key Pair Address	Tree Height = 0	Tree Index

Figure 8: BPORS+FP child secret generation address.

4.4 CS-WOTS$^+$ Public Key Generation (Function `wots_PKgen`)

A CS-WOTS$^+$ key pair defines a virtual structure that consists of `len` hash chains of length w. Each of the `len` strings of n bytes in the private key defines the start node for one hash chain. The public key is the tweakable hash of the end nodes of these hash chains. To compute the hash chains, the chaining function (Algorithm 2) is used. A CS-WOTS$^+$ hash address `ADRS` and a seed `PK.seed` have to be provided by the calling algorithm as well as a secret seed `SK.seed`. The address `ADRS` MUST encode the address of the CS-WOTS$^+$ key pair within the Kylin structure. Hence, a CS-WOTS$^+$ algorithm MUST NOT manipulate any parts of `ADRS` other than the last three 32-bit words. Note that the `PK.seed` used here is public information also available to a verifier.

The following pseudocode (Algorithm 4) describes an algorithm for generating the public key `PK`.

Algorithm 4: `wots_PKgen` — Generating a CS-WOTS$^+$ public key.

```
# Input: secret seed SK.seed, address ADRS, public seed PK.seed
# Output: CS-WOTS+ public key pk
wots_PKgen(SK.seed, PK.seed, ADRS) {
  wotspkADRS = ADRS; // copy address to create OTS public key address
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
  skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk[i] = PRF(SK.seed, skADRS);
    ADRS.setChainAddress(i);
    ADRS.setHashAddress(0);
    tmp[i] = chain(sk[i], 0, w - 1, PK.seed, ADRS);
  }
  wotspkADRS.setType(WOTS_PK);
  wotspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk = T_len(PK.seed, wotspkADRS, tmp);
  return pk;
}
```

4.5 CS-WOTS$^+$ Encoding (Function `wots_encode`)

Let us explain the encoding algorithm (Algorithm 5). The problem can be divided into several subproblems by considering the value of the first element c_{l-i}. To encode a natural number $x \in [0, D_{i,m})$, we can simply determine $c_{l-i} = j$ by seeking which j satisfies $x \in [\sum_{k<j} D_{i-1,m-k}, \sum_{k\leq j} D_{i-1,m-k})$. Once the value of v_{l-i} is determined, we proceed to its next terms until all elements are decided. All arithmetic operations MUST be compute in unsigned 256-bit. Array D can be precomputed to avoid computing D for each encoding function call.

Algorithm 5: `wots_encode` — Encoding message to constant-sum encoding.

```
# Input: Message M
# Output: Constant-Sum Encoding of M
wots_encode(M) {

  s=floor(len*(w-1)/2)
  Let D be an 2d array of size (len+1)*(s+1) initialized to 0

  D[0][0]=1
  for( i = 1 ; i <= len ; i++ ){
    for( j = 0 j <= s ; j++ ){
      for( k = 0 ; k < w and k <= j ; k++ ){
        D[i][j] = D[i][j] + D[i-1][j-k]
      }
    }
  }
  // array D can be precomputed

  Let c be an array of size len
  for( i = len ; i >= 1 ; i-- ){
    for( j = 0 ; j < w and j <= s  ; j++ ){
      if( M >= D[i-1][s-j] ){
        M = M - D[i-1][s-j]
      } else {
        c[len-i] = j
        break
      }
    }
    s = s - c[len-i]
  }

  return c;
}
```

4.6 CS-WOTS$^+$ Signature Generation (Function `wots_sign`)

A CS-WOTS$^+$ signature is a length `len` array of n-byte strings. The CS-WOTS$^+$ signature is first generated by mapping a message M to `len` integers between 0 and $w-1$, with their sum equal to $\lfloor \texttt{len} \cdot (w-1)/2 \rfloor$. A CS-WOTS$^+$ hash address `ADRS`, a public seed `PK.seed`, and a secret seed `SK.seed` have to be provided by the calling algorithm. The address will encode the address of the CS-WOTS$^+$ key pair within a greater structure. Hence, a CS-WOTS$^+$ algorithm MUST NOT manipulate any parts of `ADRS` other than the last three 32-bit words. Note that the `PK.seed` used here is public information also available to a verifier while the secret seed `SK.seed` is private information. The pseudocode for generating a CS-WOTS$^+$ signature `SIG` is shown below (Algorithm 6).

Algorithm 6: `wots_sign` — Generating a CS-WOTS$^+$ signature on a message M.

```
# Input: Message M, secret seed SK.seed, public seed PK.seed, address ADRS
# Output: CS-WOTS+ signature sig
wots_sign(M, SK.seed, PK.seed, ADRS) {
  msg = wots_encode(M);
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
    skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk = PRF(SK.seed, skADRS);
    ADRS.setChainAddress(i);
    ADRS.setHashAddress(0);
    sig[i] = chain(sk, 0, msg[i], PK.seed, ADRS);
  }
  return sig;
}
```

The data format for a signature is given in Fig. 9.

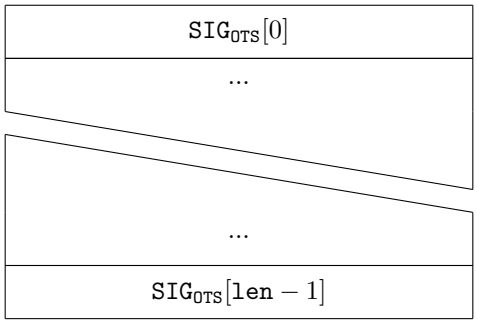


Figure 9: CS-WOTS$^+$ Signature Data Format.

4.7 CS-WOTS$^+$ Public-Key Recovery (Function `wots_pkFromSig`)

Kylin uses implicit signature verification for CS-WOTS$^+$. In order to verify a CS-WOTS$^+$ signature `SIG` on a message M, the verifier computes a CS-WOTS$^+$ public key value from the signature. This can be done by "completing" the chain computations starting from the signature values, using the constant-sum base-w encoding values derived from the message hash. This step, called `wots_pkFromSig`, is described below in Algorithm 7. The result of `wots_pkFromSig` then has to be verified. In a standalone version, this would be done by simple comparison. When used in Kylin the output value is verified by using it to compute a Kylin public key.

A CS-WOTS$^+$ hash address `ADRS` and a public seed `PK.seed` have to be provided by the calling algorithm. The address will encode the address of the CS-WOTS$^+$ key pair within the Kylin structure.

Hence, a CS-WOTS$^+$ algorithm MUST NOT manipulate any parts of `ADRS` other than the last three 32-bit words. Note that the `PK.seed` used here is public information also available to a verifier.

Algorithm 7: `wots_pkFromSig` — Computing a CS-WOTS$^+$ public key from a message and its signature.

```
# Input: Message M, CS-WOTS+ signature sig, address ADRS, public seed PK.seed
# Output: CS-WOTS+ public key pk_sig derived from sig
wots_pkFromSig(sig, M, PK.seed, ADRS) {
  wotspkADRS = ADRS;
  msg = wots_encode(M)
  for ( i = 0; i < len; i++ ) {
    ADRS.setChainAddress(i);
    tmp[i] = chain(sig[i], msg[i], w - 1 - msg[i], PK.seed, ADRS);
  }
  wotspkADRS.setType(WOTS_PK);
  wotspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk_sig = T_len(PK.seed, wotspkADRS, tmp);
  return pk_sig;
}
```

5 The Kylin Hypertree

In this section, we explain how the Kylin hypertree is built. We first explain how CS-WOTS$^+$ gets combined with a binary hash tree, leading to a fixed input-length version of the eXtended Merkle Signature Scheme (XMSS). Afterwards, we explain how to go to a hypertree from there. The hypertree might be viewed as a fixed input-length version of multi-tree XMSS (XMSS^{MT}).

5.1 (Fixed Input-Length) XMSS

XMSS is a method for signing a potentially large but fixed number of messages. It is based on the Merkle signature scheme. It authenticates $2^{h_{\text{XMSS}}}$ CS-WOTS$^+$ public keys using a binary tree of height h_{XMSS}. Hence, an XMSS key pair for height h_{XMSS} can be used to sign 2 different messages. Each node in the binary tree is an n-byte value which is the tweakable hash of the concatenation of its two child nodes. The leaves are the CS-WOTS$^+$ public keys. The XMSS public key is the root node of the tree. In Kylin, the XMSS secret key is the single secret seed that is used to generate all CS-WOTS$^+$ secret keys.

An XMSS signature in the context of Kylin consists of the CS-WOTS$^+$ signature on the message and the so-called authentication path. The latter is a vector of tree nodes that allow a verifier to compute a value for the root of the tree starting from a CS-WOTS$^+$ signature. A verifier computes the root value and verifies its correctness. A standalone XMSS signature also contains the index of the used CS-WOTS$^+$ key pair. In the context of Kylin this is not necessary as the Kylin signature allows to compute the index for each XMSS signature contained.

5.1.1 XMSS Parameters

XMSS has the following parameters:

- h_{XMSS} : the height (number of levels $-$ 1) of the tree.
- n: the length in bytes of messages as well as of each node.
- w : the Winternitz parameter as defined for CS-WOTS$^+$ in the previous Section.

There are $2^{h_{\text{XMSS}}}$ leaves in the tree. XMSS signatures are denoted by $\texttt{SIG}_{\texttt{XMSS}}$ (`SIG_XMSS` in pseudocode). CS-WOTS$^+$ signatures are denoted by `SIG`.

XMSS parameters are implicitly included in algorithm inputs as needed.

5.1.2 XMSS Private Key

In the context of Kylin, an XMSS private key is the single secret seed `SK.seed` contained in the Kylin secret key. It is used to generate the CS-WOTS$^+$ secret keys within the structure of an XMSS key pair as described in Section 4.

5.1.3 TreeHash (Function `treehash`)

For the computation of the internal n-byte nodes of a Merkle tree, the subroutine `treehash` (Algorithm 8) accepts a secret seed `SK.seed`, a public seed `PK.seed`, an unsigned integer s (the start index), an unsigned integer z (the target node height), and an address `ADRS` that encodes the address of the containing tree. For the height of a node within a tree, counting starts with the leaves at height zero. The `treehash` algorithm returns the root node of a tree of height z with the leftmost leaf being the CS-WOTS$^+$ pk at index s. It is REQUIRED that $s \,\%\, 2^z = 0$, i.e. that the leaf at index s is a leftmost leaf of a sub-tree of height z. Otherwise the algorithm fails as it would compute non-existent nodes. The `treehash` algorithm described here uses a stack holding up to $(z-1)$ nodes, with the usual stack functions `push()` and `pop()`. We furthermore assume that the height of a node (an unsigned integer) is stored alongside a node's value (an n-byte string) on the stack.

Algorithm 8: `treehash` — The TreeHash algorithm.

```
# Input: Secret seed SK.seed, start index s, target node height z, public seed
↪  PK.seed, address ADRS
# Output: n-byte root node - top node on Stack
treehash(SK.seed, s, z, PK.seed, ADRS) {
  if( s % (1 << z) != 0 ) return -1;
  for ( i = 0; i < 2^z; i++ ) {
    ADRS.setType(WOTS_HASH);
    ADRS.setKeyPairAddress(s + i);
    node = wots_PKgen(SK.seed, PK.seed, ADRS);
    ADRS.setType(TREE);
    ADRS.setTreeHeight(1);
    ADRS.setTreeIndex(s + i);
    while ( Top node on Stack has same height as node ) {
      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
      node = H(PK.seed, ADRS, (Stack.pop()  node));
      ADRS.setTreeHeight(ADRS.getTreeHeight() + 1);
    }
    Stack.push(node);
  }
  return Stack.pop();
}
```

5.1.4 XMSS Public Key Generation (Function `xmss_PKgen`)

The XMSS public key is computed as described in `xmss_PKgen` (Algorithm 9). In the context of Kylin the XMSS public key PK is the root of the binary hash tree. The root is computed using `treehash`. The public key generation takes a secret seed `SK.seed`, a public seed `PK.seed`, and an address `ADRS`. The latter encodes the position of this XMSS instance within the Kylin structure.

5.1.5 XMSS Signature

An XMSS signature is a $((\texttt{len} + h_{\text{XMSS}}) \cdot n)$-byte string consisting of

- a CS-WOTS$^+$ signature $\texttt{SIG}_{\texttt{OTS}}$ taking $\texttt{len} \cdot n$ bytes,
- the authentication path `AUTH` for the leaf associated with the used CS-WOTS$^+$ key pair taking $h_{\text{XMSS}} \cdot n$ bytes.

Algorithm 9: `xmss_PKgen` — Generating an XMSS public key.

```
# Input: Secret seed SK.seed, public seed PK.seed, address ADRS
# Output: XMSS public key PK
xmss_PKgen(SK.seed, PK.seed, ADRS) {
  pk = treehash(SK.seed, 0, h_XMSS, PK.seed, ADRS)
  return pk;
}
```

The authentication path is an array of h_{XMSS} n-byte strings. It contains the siblings of the nodes in on the path from the used leaf to the root. It does not contain the nodes on the path itself. These nodes in `AUTH` are needed by a verifier to compute a root node for the tree from a CS-WOTS$^+$ public key. A node N is addressed by its position in the tree. $N(x, y)$ denotes the y-th node on level x with $y = 0$ being the leftmost node on a level. The leaves are on level 0, the root is on level h_{XMSS}. An authentication path contains exactly one node on every layer $0 \leq x \leq (h_{\text{XMSS}} - 1)$. For the i-th CS-WOTS$^+$ key pair, counting from zero, the j-th authentication path node is

$$\texttt{AUTH}[j] = N\left(j, \lfloor \frac{i}{2^j} \rfloor \oplus 1\right) \quad .$$

The computation of the authentication path is discussed in Section 5.1.6. The data format for a signature is given in Fig. 10.

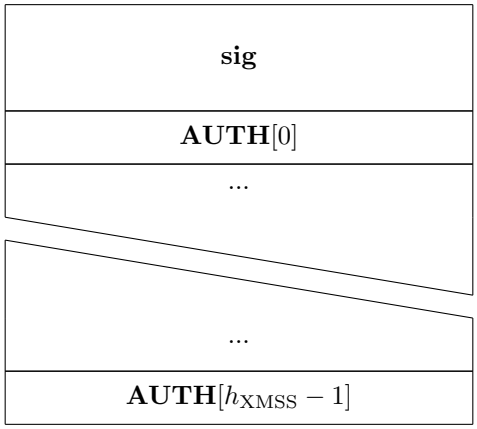


Figure 10: XMSS Signature

5.1.6 XMSS Signature Generation (Function `xmss_sign`)

To compute the XMSS signature of a message M in the context of Kylin, the secret seed `SK.seed`, the public seed `PK.seed`, the index `idx` of the CS-WOTS$^+$ key pair to be used, and the address `ADRS` of the XMSS instance are needed. First, a CS-WOTS$^+$ signature of the message digest is computed using the CS-WOTS$^+$ instance at index `idx`. Next, the authentication path is computed.

The node values of the authentication path MAY be computed in any way. The least memory-intensive method is to compute all nodes using the `treehash` algorithm (Algorithm 8). This is described here. Note that the details of how this step is implemented are not relevant to interoperability; it is not necessary to know any of these details in order to perform the signature verification operation.

5.1.7 XMSS Compute Public Key from Signature (Function `xmss_pkFromSig`)

Kylin makes use of implicit signature verification of XMSS signatures. An XMSS signature is used to compute a candidate XMSS public key, i.e., the root of the tree. This is used in further computations (signature of the tree above) and implicitly verified by the outcome of that computation. Hence, this specification does not contain an `xmss_verify` method but the method `xmss_pkFromSig`.

Algorithm 10: `xmss_sign` — Generating an XMSS signature.

```
# Input: n-byte message M, secret seed SK.seed, index idx, public seed PK.seed,
  ↪  address ADRS
# Output: XMSS signature SIG_XMSS = (sig || AUTH)
xmss_sign(M, SK.seed, idx, PK.seed, ADRS) {
  // build authentication path
  for ( j = 0; j < h_XMSS; j++ ) {
    k = floor(idx / (2^j)) XOR 1;
    AUTH[j] = treehash(SK.seed, k * 2^j, j, PK.seed, ADRS);
  }
  ADRS.setType(WOTS_HASH);
  ADRS.setKeyPairAddress(idx);
  sig = wots_sign(M, SK.seed, PK.seed, ADRS);
  SIG_XMSS = sig  AUTH;
  return SIG_XMSS;
}
```

The method `xmss_pkFromSig` takes an n-byte message M, an XMSS signature $\texttt{SIG}_{\texttt{XMSS}}$, a signature index `idx`, a public seed `PK.seed`, and an address `ADRS`. The latter encodes the position of the current XMSS instance within the virtual structure of the Kylin key pair. First, `wots_pkFromSig` is used to compute a candidate CS-WOTS$^+$ public key. This in turn is used together with the authentication path to compute a root node which is then returned. The algorithm `xmss_pkFromSig` is given as Algorithm 11.

5.2 HT: The Hypertree

The Kylin hypertree HT is a variant of XMSS^{MT}. It is essentially a certification tree of XMSS instances. A HT is a tree of several layers of XMSS trees. The trees on top and intermediate layers are used to sign the public keys, i.e., the root nodes, of the XMSS trees on the respective next layer below. Trees on the lowest layer are used to sign the actual messages, which are BPORS+FP public keys in Kylin.

Consider a HT of total height h that has d layers of XMSS trees with per-layer heights $(h_0, \ldots, h_{d-1})$ from bottom to top, where $\sum_{i=0}^{d-1} h_i = h$. Then layer $d-1$ contains one XMSS tree. In general, layer i contains $2^{\sum_{j=i+1}^{d-1} h_j}$ XMSS trees. Finally, layer 0 contains 2^{h-h_0} XMSS trees.

5.2.1 HT Parameters

In addition to all XMSS parameters, a HT requires the hypertree height h, the number of tree layers d, and per-layer XMSS heights $(h_0, \ldots, h_{d-1})$ with positive integers satisfying $\sum_{i=0}^{d-1} h_i = h$. The same Winternitz parameter w is used for all tree layers.

5.2.2 HT Key Generation (Function `ht_PKgen`)

The HT private key is the secret seed `SK.seed` which is used to generate all the CS-WOTS$^+$ private keys within the virtual structure spanned by the HT.

The HT public key is the public key (root node) of the single XMSS tree on the top layer. Its computation is explained below. The public key generation takes as input a private and a public seed.

5.2.3 HT Signature

A HT signature $\texttt{SIG}_{\texttt{HT}}$ is a byte string of length $(h + d \cdot \texttt{len}) \cdot n$. It consists of d XMSS signatures, where layer i contributes one XMSS signature of $(h_i + \texttt{len}) \cdot n$ bytes.

The data format for a signature is given in Fig. 11.

5.2.4 HT Signature Generation (Function `ht_sign`)

To compute a HT signature $\texttt{SIG}_{\texttt{HT}}$ of a message M, `ht_sign` (Algorithm 13) uses `xmss_sign` as defined in Section 5.1.6. The algorithm `ht_sign` takes as input a message M, a private seed `SK.seed`, a public

Algorithm 11: `xmss_pkFromSig` — Computing an XMSS public key from an XMSS signature.

```
# Input: index idx, XMSS signature SIG_XMSS = (sig || AUTH), n-byte message M,
↪  public seed PK.seed, address ADRS
# Output: n-byte root value node[0]
xmss_pkFromSig(idx, SIG_XMSS, M, PK.seed, ADRS){ // compute WOTS+ pk from WOTS+
↪  sig
ADRS.setType(WOTS_HASH);
ADRS.setKeyPairAddress(idx);
sig = SIG_XMSS.getWOTSSig();
AUTH = SIG_XMSS.getXMSSAUTH();
node[0] = wots_pkFromSig(sig, M, PK.seed, ADRS);
// compute root from WOTS+ pk and AUTH
ADRS.setType(TREE);
ADRS.setTreeIndex(idx);
for ( k = 0; k < h_XMSS; k++ ) {
  ADRS.setTreeHeight(k+1);
  if ( (floor(idx / (2^k)) % 2) == 0 ) {
      ADRS.setTreeIndex(ADRS.getTreeIndex() / 2);
      node[1] = H(PK.seed, ADRS, (node[0]  AUTH[k]));
    } else {
      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
      node[1] = H(PK.seed, ADRS, (AUTH[k]  node[0]));
    }
    node[0] = node[1];
  }
  return node[0];
}
```

Algorithm 12: `ht_PKgen` — Generating an HT public key.

```
# Input: Private seed SK.seed, public seed PK.seed
# Output: HT public key PK_HT
ht_PKgen(SK.seed, PK.seed){
     ADRS = toByte(0, 32);
     ADRS.setLayerAddress(d-1);
     ADRS.setTreeAddress(0);
     root = xmss_PKgen(SK.seed, PK.seed, ADRS);
     return root;
}
```

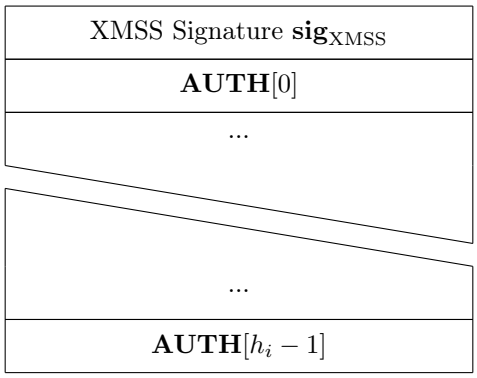


Figure 11: HT Signature.

seed `PK.seed`, and an index `idx`. The index identifies the leaf of the hypertree to be used to sign the message. The HT signature then consists of a stack of XMSS signatures using the XMSS trees on the path from the leaf with index idx to the top tree. Note that idx is passed as two separate arguments, split into an index to address the specific tree and the leaf index within that tree. This allows for a somewhat higher hypertree, as one can use a 64-bit integer for `tree_idx` to support parameters that conform to $h < 64 + h_0$. This matches the parameters in this specification. If other parameter sets are used that allow greater h, the data type of `tree_idx` MUST be adapted accordingly.

Algorithm `ht_sign` uses `xmss_pkFromSig` to compute the root node of an XMSS instance after that instance was used for signing. An alternative is to use `xmss_PKgen`. However, `xmss_PKgen` rebuilds the whole tree while `xmss_pkFromSig` only does one call to `wots_pkFromSig` and $(h_i - 1)$ calls to **H** on layer i. The algorithm `ht_sign` as described below is just one way to generate a HT signature. Other methods MAY be used as long as they generate the same output.

Algorithm 13: `ht_sign` — Generating an HT signature.

```
# Input: Message M, private seed SK.seed, public seed PK.seed, tree index
  ↪ idx_tree, leaf index idx_leaf
# Output: HT signature SIG_HT
ht_sign(M, SK.seed, PK.seed, idx_tree, idx_leaf) {
  // init
  // per-layer XMSS heights are h_0, h_1, ..., h_{d-1} with sum h
  ADRS = toByte(0, 32);
  // sign
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  SIG_tmp = xmss_sign(M, SK.seed, idx_leaf, PK.seed, ADRS);
  SIG_HT = SIG_HT  SIG_tmp;
  root = xmss_pkFromSig(idx_leaf, SIG_tmp, M, PK.seed, ADRS);
  for ( j = 1; j < d; j++ ) {
    idx_leaf = h_j least significant bits of idx_tree;
    idx_tree = idx_tree >> h_j;
    ADRS.setLayerAddress(j);
    ADRS.setTreeAddress(idx_tree);
    SIG_tmp = xmss_sign(root, SK.seed, idx_leaf, PK.seed, ADRS);
    SIG_HT = SIG_HT  SIG_tmp;
    if ( j < d - 1 ) {
      root = xmss_pkFromSig(idx_leaf, SIG_tmp, root, PK.seed, ADRS);
    }
  }
  return SIG_HT;
}
```

5.2.5 HT Signature Verification (Function `ht_verify`)

HT signature verification (Algorithm 14) can be summarized as d calls to `xmss_pkFromSig` and one comparison with a given value. HT signature verification takes a message M, a signature $\mathtt{SIG}_{\mathtt{HT}}$, a public seed `PK.seed`, an index `idx` (split into a tree index and a leaf index, as above), and a HT public key $\mathtt{PK}_{\mathtt{HT}}$.

Algorithm 14: `ht_verify` — Verifying a HT signature $\mathbf{SIG_{HT}}$ on a message M using a HT public key $\mathbf{PK}_{HT}$.

```
# Input: Message M, signature SIG_HT, public seed PK.seed, tree index idx_tree,
↪  leaf index idx_leaf, HT public key PK_HT.
# Output: Boolean
ht_verify(M, SIG_HT, PK.seed, idx_tree, idx_leaf, PK_HT){
  // init
  // per-layer XMSS heights are h_0, h_1, ..., h_{d-1} with sum h
  ADRS = toByte(0, 32);
  // verify
  SIG_tmp = SIG_HT.getXMSSSignature(0);
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  node = xmss_pkFromSig(idx_leaf, SIG_tmp, M, PK.seed, ADRS);
  for ( j = 1; j < d; j++ ) {
    idx_leaf = h_j least significant bits of idx_tree;
    idx_tree = idx_tree >> h_j;
    SIG_tmp = SIG_HT.getXMSSSignature(j);
    ADRS.setLayerAddress(j);
    ADRS.setTreeAddress(idx_tree);
    node = xmss_pkFromSig(idx_leaf, SIG_tmp, node, PK.seed, ADRS);
    }
  if ( node == PK_HT ) {
    return true;
  } else {
    return false;
  }
}
```

6 BPORS+FP: Bucket-Thinned Few-Time Signatures

The Kylin hypertree HT is not used to sign the original message directly. Instead, the randomized message hash selects a bottom HT leaf and gives an n-byte digest `md`. Kylin signs this digest with BPORS+FP, a bucket-thinned few-time signature component based on the construction in [28] and on forced-pruning style size control [1]. The resulting BPORS+FP public key is then signed by the bottom XMSS tree of the hypertree.

BPORS+FP has an outer bucketing layer and a child PORS layer. The outer layer contains K coordinates. Each coordinate contains 2^B buckets, and each bucket stores the public key of an independent child PORS instance. A child PORS instance has t leaves and opens a k-subset of those leaves using the Octopus authentication set of the left-filled Merkle tree on t leaves. Kylin uses a global forced-pruning threshold $m_{\max}$: the signer repeats the BPORS encoding hash with an explicit counter until the total authentication node count, including the KB outer authentication nodes and all child Octopus nodes, is at most $m_{\max}$.

6.1 BPORS+FP Parameters

BPORS+FP uses the parameters $K, B, t, k, m_{\max}$ and a counter length c_{ctr} in bytes:

- K: the number of outer coordinates and selected child PORS keys.

- B: the bucket-address bit length; each coordinate has 2^B buckets.
- t: the number of leaves in each child PORS tree. This value need not be a power of two; non-power-of-two trees use the left-filled tree convention.
- k: the number of leaves opened inside each selected child PORS key.
- $m_{\max}$: the fixed global authentication-node budget.
- c_{ctr}: the number of bytes used to encode the rejection counter. The packaged implementation uses $c_{\mathrm{ctr}} = 8$.

For a digest $\mathtt{md}$ and counter value $\mathtt{ctr}$, a domain-separated XOF maps $(\mathtt{md}, \mathtt{ctr}, \mathtt{PK.seed}, \mathtt{ADRS})$ to

$$((j_0, I_0), \ldots, (j_{K-1}, I_{K-1})) \in \left([2^B] \times \binom{[t]}{k}\right)^K.$$

Here j_c is the bucket selected in coordinate c, and $I_c \subseteq [t]$ is the k-subset opened in the corresponding child tree. The candidate is accepted when

$$A_{\mathrm{tot}} = KB + \sum_{c=0}^{K-1} |\mathrm{Oct}_t(I_c)| \leq m_{\max}.$$

6.2 BPORS+FP Private Key and Public Key

As with the rest of Kylin, BPORS+FP private values are generated pseudorandomly from the single seed $\mathtt{SK.seed}$. The secret value for coordinate c, bucket j, and child leaf u is derived as

$$x_{c,j,u} \leftarrow \mathbf{PRF}(\mathtt{SK.seed}, \mathtt{ADRS}_{c,j,u}),$$

where the address type and the coordinate, bucket, and child-leaf fields separate this value from all WOTS and hypertree values. The child leaf is $\mathbf{F}(\mathtt{PK.seed}, \mathtt{ADRS}_{c,j,u}, x_{c,j,u})$.

Each child PORS public key is the root of the left-filled Merkle tree over its t child leaves. For each outer coordinate c, the 2^B child public keys are placed as leaves of a complete binary Merkle tree of height B. The BPORS+FP public key is

$$\mathtt{PK}_{\mathtt{BPORS}} = \mathbf{T}_K(\mathtt{PK.seed}, \mathtt{ADRS}_{\mathtt{BPORS_PK}}, \mathtt{root}_0 \| \cdots \| \mathtt{root}_{K-1}),$$

where $\mathtt{root}_c$ is the root of the outer tree for coordinate c.

6.3 BPORS+FP Signature Generation

A BPORS+FP signature contains the counter, the Kk selected child secret values, the child Octopus authentication nodes, the outer authentication paths, and zero padding up to the fixed budget $m_{\max}$. The signing algorithm is shown in Algorithm 15.

6.4 BPORS+FP Compute Public Key from Signature

Kylin uses implicit verification for BPORS+FP. Given $\mathtt{SIG}_{\mathtt{BPORS}}$ and $\mathtt{md}$, the verifier re-derives the accepted bucket labels and child subsets from the counter, reconstructs each child public key from the revealed secrets and Octopus nodes, reconstructs each outer root using the outer authentication path, checks that unused authentication slots are zero, and finally compresses the K outer roots. This is shown in Algorithm 16.

7 The Full Kylin Signature Scheme

We now have all ingredients to describe our main construction Kylin. It keeps the SPHINCS-style hypertree structure, but uses CS-WOTS$^+$ one-time signatures and BPORS+FP as the bottom few-time signature component.

Algorithm 15: `bpors_sign` — Generate a BPORS+FP signature and public key.

```
# Input: n-byte string M, secret seed SK.seed, address ADRS, public seed
↪ PK.seed
# Output: BPORS+FP signature SIG_BPORS and public key PK_BPORS
bpors_sign(M, SK.seed, PK.seed, ADRS) {
  ctr = 0;
  do {
    (bucket[0..K-1], I[0..K-1]) = bpors_encode(M, ctr, PK.seed, ADRS);
    A_tot = K*B;
    for (c = 0; c < K; c++) {
      AUTH_child[c] = Octopus(t, I[c]);
      A_tot = A_tot + size(AUTH_child[c]);
    }
    ctr = ctr + 1;
  } while (A_tot > m_max);

  SIG_BPORS = SIG_BPORS  toByte(ctr - 1, c_ctr);
  for (c = 0; c < K; c++) {
    for (j = 0; j < k; j++) {
      SIG_BPORS = SIG_BPORS  bpors_SKgen(SK.seed, ADRS, c, bucket[c], I[c][j]);
    }
  }

  for (c = 0; c < K; c++) {
    (PK_child[c], AUTH_values) =
        bpors_child_open(SK.seed, PK.seed, ADRS, c, bucket[c], I[c]);
    SIG_BPORS = SIG_BPORS  AUTH_values;
    (root[c], AUTH_outer) =
        bpors_outer_open(SK.seed, PK.seed, ADRS, c, bucket[c]);
    SIG_BPORS = SIG_BPORS  AUTH_outer;
  }

  SIG_BPORS = SIG_BPORS  zero padding to m_max authentication nodes;
  bporspkADRS = ADRS;
  bporspkADRS.setType(BPORS_ROOTS);
  PK_BPORS = T_K(PK.seed, bporspkADRS, root);
  return (SIG_BPORS, PK_BPORS);
}
```

Algorithm 16: `bpors_pkFromSig` — Compute a BPORS+FP public key from a signature.

```
# Input: BPORS+FP signature SIG_BPORS, n-byte string M, public seed PK.seed,
  ↪ address ADRS
# Output: BPORS+FP public key PK_BPORS, or fail
bpors_pkFromSig(SIG_BPORS, M, PK.seed, ADRS) {
  ctr = SIG_BPORS.getCounter();
  (bucket[0..K-1], I[0..K-1]) = bpors_encode(M, ctr, PK.seed, ADRS);
  A_tot = K*B;
  for (c = 0; c < K; c++) {
    AUTH_child[c] = Octopus(t, I[c]);
    A_tot = A_tot + size(AUTH_child[c]);
  }
  if (A_tot > m_max) {
    return fail;
  }

  for (c = 0; c < K; c++) {
    for (j = 0; j < k; j++) {
      sk[j] = SIG_BPORS.getSK(c, j);
      leaf[j] = F(PK.seed, childTreeADRS(ADRS, c, bucket[c], I[c][j]), sk[j]);
    }
    PK_child[c] =
        bpors_child_rootFromSig(leaf, SIG_BPORS.getChildAUTH(c), I[c], PK.seed,
        ↪ ADRS);
    root[c] =
        bpors_outer_rootFromSig(PK_child[c], SIG_BPORS.getOuterAUTH(c),
                                bucket[c], PK.seed, ADRS);
  }

  if (SIG_BPORS.paddingIsNonzero()) {
    return fail;
  }
  bporspkADRS = ADRS;
  bporspkADRS.setType(BPORS_ROOTS);
  PK_BPORS = T_K(PK.seed, bporspkADRS, root);
  return PK_BPORS;
}
```

7.1 Kylin Parameters

Kylin has the following parameters:

- n: the security parameter in bytes.
- w: the Winternitz parameter as defined in Section 4.1.
- h: the height of the hypertree as defined in Section 5.2.1.
- d: the number of layers in the hypertree as defined in Section 5.2.1.
- $K, B, t, k, m_{\max}$: the BPORS+FP parameters as defined in Section 6.1.

All the restrictions stated in the previous sections apply. Moreover, from these values the values m and `len` are computed as

- m: the message digest length in bytes. It is computed as

$$m = n + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor.$$

  The first n bytes are the BPORS+FP digest. The remaining bytes encode the hypertree tree and leaf indices.

- `len`: the number of n-byte string elements in a CS-WOTS$^+$ private key, public key, and signature. The value `len` is chosen so that the size of the following poset is larger than the desired message space $\{0,1\}^{8n}$ (cf. Section 2).

$$\left| D_{\texttt{len}, \lfloor \frac{\texttt{len} \cdot (w-1)}{2} \rfloor} \right| \geq 2^{8n} \ .$$

In the following, we assume that all algorithms have access to these parameters.

7.2 Kylin Key Generation (Function `kylin_keygen`)

The Kylin private key contains two elements. First, the n-byte secret seed `SK.seed` which is used to generate all the CS-WOTS$^+$ and BPORS+FP private key elements. Second, an n-byte PRF key `SK.prf` which is used to deterministically generate a randomization value for the randomized message hash.

The Kylin public key also contains two elements. First, the HT public key, i.e. the root of the tree on the top layer. Second, an n-byte public seed value `PK.seed` which is sampled uniformly at random.

As `kylin_sign` does not get the public key, but needs access to `PK.seed` (and possibly to `PK.root` for fault attack mitigation), the Kylin secret key contains a copy of the public key.

The description of algorithm `kylin_keygen` assumes the existence of a function `sec_rand` which on input i returns i-bytes of cryptographically strong randomness.

Algorithm 17: `kylin_keygen` — Generate a Kylin key pair.

```
# Input: (none)
# Output: Kylin key pair (SK,PK)
kylin_keygen( ){
  SK.seed = sec_rand(n);
  SK.prf = sec_rand(n);
  PK.seed = sec_rand(n);
  PK.root = ht_PKgen(SK.seed, PK.seed);
  return ( (SK.seed, SK.prf, PK.seed, PK.root), (PK.seed, PK.root) );
}
```

The format of a Kylin private and public key is given in Fig. 12.

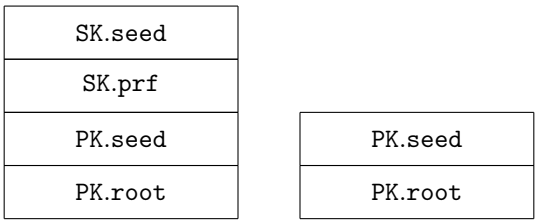


Figure 12: Left: Kylin secret key. Right: Kylin public key.

7.3 Kylin Signature

A Kylin signature SIG is a byte string of length

$$n + c_{\text{ctr}} + (Kk + m_{\max})n + (h + d \cdot \texttt{len})n.$$

It consists of an n-byte randomization string R, a BPORS+FP signature $\texttt{SIG}_{\texttt{BPORS}}$ consisting of an explicit counter, Kk n-byte secret values and $m_{\max}$ n-byte authentication-node slots, and a HT signature $\texttt{SIG}_{\texttt{HT}}$ of $(h + d \cdot \texttt{len})n$ bytes.

The data format for a signature is given in Fig. 13.

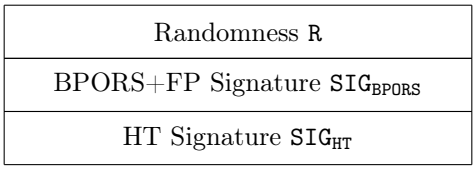


Figure 13: Kylin signature.

7.4 Kylin Signature Generation (Function kylin_sign)

Generating a Kylin signature consists of four steps. First, a random value R is pseudorandomly generated. Next, this is used to compute an m byte message digest which is split into an n-byte BPORS+FP digest md, a $\lfloor (h - h_0 + 7)/8 \rfloor$-byte tree index tmp_idx_tree, and a $\lfloor (h_0 + 7)/8 \rfloor$-byte leaf index tmp_idx_leaf. The values idx_tree and idx_leaf are computed by extracting the necessary number of bits. The digest md is then signed with the BPORS+FP key pair associated with the idx_leaf-th bottom leaf of the idx_tree-th XMSS tree on the lowest HT layer. The resulting BPORS+FP public key is then signed using HT. The index is never actually used as a whole, but immediately split into a tree index and a leaf index, for ease of implementation.

When computing R, the PRF takes a n-byte string opt which is initialized with PK.seed but can be overwritten with randomness if the global variable RANDOMIZE is set. This option is given as otherwise Kylin signatures would be always deterministic.

7.5 Kylin Signature Verification (Function kylin_verify)

Kylin signature verification (Algorithm 19) can be summarized as recomputing the message digest and index, computing a candidate BPORS+FP public key, and verifying the HT signature on that public key. Note that the HT signature verification will fail if the BPORS+FP public key is not matching the real one (with overwhelming probability). Kylin signature verification takes a message M, a signature SIG, and a Kylin public key PK.

8 Parameter Sets and Hash/XOF Instantiation

This section discusses instantiations for Kylin. Kylin can be viewed as a signature template. It is a way to build a signature scheme by instantiating the cryptographic function families used. We consider different ways to implement the cryptographic function families as different signature systems. Orthogonal to instantiating the cryptographic function families are parameter sets. Parameter sets assign specific values to the Kylin parameters described below.

Algorithm 18: `kylin_sign` — Generating a Kylin signature.

```
# Input: Message M, private key SK = (SK.seed, SK.prf, PK.seed, PK.root)
# Output: Kylin signature SIG
kylin_sign(M, SK){ // init
ADRS = toByte(0, 32);
  // generate randomizer
  opt = PK.seed;
  if(RANDOMIZE){
    opt = rand(n);
  }
  R = PRF_msg(SK.prf, opt, M);
  SIG = SIG  R;
  // compute message digest and index
  digest = H_msg(R, PK.seed, PK.root, M);
  md = first n bytes of digest;
  tmp_idx_tree = next floor((h - h_0 +7)/ 8) bytes of digest;
  tmp_idx_leaf = next floor((h_0 +7)/ 8) bytes of digest;
  idx_tree = first h - h_0 bits of tmp_idx_tree;
  idx_leaf = first h_0 bits of tmp_idx_leaf;
  // BPORS+FP sign
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  ADRS.setType(BPORS_CHILD_TREE);
  ADRS.setKeyPairAddress(idx_leaf);
  (SIG_BPORS, PK_BPORS) = bpors_sign(md, SK.seed, PK.seed, ADRS);
  SIG = SIG  SIG_BPORS;
  // sign BPORS+FP public key with HT
  ADRS.setType(TREE);
  SIG_HT = ht_sign(PK_BPORS, SK.seed, PK.seed, idx_tree, idx_leaf);
  SIG = SIG  SIG_HT;
  return SIG;
}
```

In this section, we first define the requirements on parameters and discuss existing trade-offs between security, sizes, and speed controlled by the different parameters. Then we summarize the frozen NGCC-facing parameter sets and the hash/XOF instantiation used by the packaged implementation.

8.1 Kylin Parameter Sets

Kylin is described by the following parameters already described in the previous sections. All parameters take positive integer values.

- n: the security parameter in bytes.
- w : the Winternitz parameter.
- h: the height of the hypertree.
- d: the number of layers in the hypertree.
- $K, B, t, k, m_{\max}$: the BPORS+FP few-time parameters.

Moreover, from these values the values m and `len` are computed as

- m: the message digest length in bytes. It is computed as

$$m = n + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor.$$

Algorithm 19: `kylin_verify` — Verify a Kylin signature **SIG** on a message M using a Kylin public key **PK**.

```
# Input: Message M, signature SIG, public key PK
# Output: Boolean
kylin_verify(M, SIG, PK){ // init
  ADRS = toByte(0, 32);
  R = SIG.getR();
  SIG_BPORS = SIG.getSIG_BPORS();
  SIG_HT = SIG.getSIG_HT();
  // compute message digest and index
  digest = H_msg(R, PK.seed, PK.root, M);
  md = first n bytes of digest;
  tmp_idx_tree = next floor((h - h_0 +7)/ 8) bytes of digest;
  tmp_idx_leaf = next floor((h_0 +7)/ 8) bytes of digest;
  idx_tree = first h - h_0 bits of tmp_idx_tree;
  idx_leaf = first h_0 bits of tmp_idx_leaf;
  // compute BPORS+FP public key
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  ADRS.setType(BPORS_CHILD_TREE);
  ADRS.setKeyPairAddress(idx_leaf);
  PK_BPORS = bpors_pkFromSig(SIG_BPORS, md, PK.seed, ADRS);
  // verify HT signature
  ADRS.setType(TREE);
  return ht_verify(PK_BPORS, SIG_HT, PK.seed, idx_tree, idx_leaf, PK.root);
}
```

- `len`: the number of n-byte string elements in a CS-WOTS$^+$ private key, public key, and signature.

$$\texttt{len} = \min\left\{\ell : \left|D_{\ell,\lfloor\frac{\ell\cdot(w-1)}{2}\rfloor}\right| \geq 2^{8n}\right\} \ ,$$

  where the size of $D_{\ell,\lfloor\frac{\ell\cdot(w-1)}{2}\rfloor}$ is evaluated using the recurrence relation in Eq. (1).

We now repeat the roles of, requirements on, and properties of these parameters. Afterwards, we give several formulas that show their exact influence on performance and security. The security parameter n is also the output length of all cryptographic function families besides $\mathbf{H_{msg}}$. Therefore, it largely determines which security level a parameter set reaches. It is also the size of virtually any node within the Kylin structure and thereby also the size of all elements in a signature, i.e., the signature size is a multiple of n.

The Winternitz parameter w determines the number and length of the hash chains per CS-WOTS$^+$ instance. A greater value for w linearly increases the length of the hash chains but logarithmically reduces their number. The number of hash chains exactly corresponds to the number of n-byte values in a CS-WOTS$^+$ signature. Thereby it largely influences the size of a Kylin signature. The product of the number and the length of hash chains directly correlates with signing speed as essentially all time in HT signature generation is spent computing CS-WOTS$^+$ public keys. Therefore, greater w means shorter signatures but slower signing. However, note the exponential gap. The bigger w gets, the more expensive is the signature size reduction. The Winternitz parameter does not influence Kylin security.

The height of the hypertree h determines the number of bottom BPORS+FP instances. Hence, it determines the probability that a bottom few-time key pair is used several times, given the number of signatures made with a Kylin key pair. Hence, the height has a direct impact on security: A taller hypertree gives more security. On the other hand, a taller tree leads to larger signatures.

The number of layers d is a pure performance trade-off parameter and does not influence security. It determines the number of layers of XMSS trees in the hypertree. The per-layer heights $(h_0, \ldots, h_{d-1})$ define the XMSS heights and must satisfy $\sum_{i=0}^{d-1} h_i = h$. Increasing d usually allows smaller subtrees and faster signing. However, d also controls the number of layers and thereby the number of CS-WOTS$^+$ signatures within a HT and thereby a Kylin signature.

The parameters $K, B, t, k, m_{\max}$ determine the performance and security of BPORS+FP. Larger B gives stronger bucket thinning because each child PORS key sees a smaller fraction of the signing history, but it also increases the cost of building the outer bucket trees and adds KB outer authentication nodes. Larger K amplifies the cover-term exponent and opens more child keys. The child parameters t and k determine the PORS cover term and the size of each child opening. The threshold $m_{\max}$ fixes the authentication-node payload: smaller values reduce signatures but increase the expected number of counter trials.

The message digest length m is the output length of $\mathbf{H_{msg}}$ in bytes. It is $n + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor$ bytes.

The number `len` of chains in a CS-WOTS$^+$ key pair determines the CS-WOTS$^+$ signature size.

8.1.1 Influence of Parameters on Security and Performance

In the following we provide formulas to compute speed, size and security for a given Kylin parameter set. This supports parameter selection. We also provide a SAGE script in Appendix A.

Key Generation. Generating the Kylin private key and `PK.seed` requires three calls to a secure random number generator. Next we have to generate the top tree (layer $d-1$). For the leaves we need to do $2^{h_{d-1}}$ CS-WOTS$^+$ key generations (`len` calls to PRF for generating the `SK` and w`len` calls to $\mathbf{F}$ for the `PK`) and we have to compress the CS-WOTS$^+$ public key (one call to $\mathbf{T}_{\texttt{len}}$). Computing the root of the top tree requires $(2^{h_{d-1}} - 1)$ calls to $\mathbf{H}$.

Signing. For randomization and message compression we need one call to $\mathbf{PRF_{msg}}$, and one to $\mathbf{H_{msg}}$. The BPORS+FP signing cost consists of counter trials, generation of the Kk revealed child secrets, construction of the selected child openings, and construction of the K outer authentication paths. In the packaged implementation this is dominated by constructing the selected outer bucket trees, i.e., about $K \cdot 2^B$ child roots, each over t child leaves, plus the child-opening work for the selected buckets. Next, we compute one HT signature which consists of d trees similar to the key generation. Hence, we have to do $\sum_{i=0}^{d-1} 2^{h_i}$ times `len` calls to $\mathbf{PRF}$ and w`len` calls to $\mathbf{F}$ as well as $\sum_{i=0}^{d-1} 2^{h_i}$ calls to $\mathbf{T}_{\texttt{len}}$. For computing the root of each tree we get additionally $\sum_{i=0}^{d-1}(2^{h_i} - 1)$ calls to $\mathbf{H}$.

Verification. First we need to compute the message hash using $\mathbf{H_{msg}}$. We need to do one BPORS+FP public-key reconstruction. This requires Kk calls to $\mathbf{F}$ to hash the revealed child secrets into leaves, one child-tree reconstruction for each selected coordinate using the supplied Octopus nodes, KB calls to $\mathbf{H}$ for the outer authentication paths, and one call to $\mathbf{T}_K$ for hashing the outer roots. Next, we have to verify d XMSS signatures which takes $< w$`len` calls to $\mathbf{F}$ and one call to $\mathbf{T}_{\texttt{len}}$ each for CS-WOTS$^+$ signature verification[2]. It also needs $\sum_{i=0}^{d-1} h_i = h$ calls to $\mathbf{H}$ for the d root computations.

The size of the Kylin private and public keys along with the signature are $4n$, $2n$, and

$$n + c_{\text{ctr}} + (Kk + m_{\max})n + (h + d \cdot \texttt{len})n$$

respectively.

The classical generic-security proxy is computed from the BPORS+FP cover term in Section 10.4; small constant factors inside the logarithm are neglected.

8.1.2 Proposed Parameter Sets and Security Levels

As explained in the previous subsection, even for a fixed security level the design of Kylin supports different tradeoffs between signature size and speed. For this submission package, we use the three NGCC-aligned Kylin instances listed in Table 3.

Table 3: Kylin parameter families.

Family	NGCC lvl	q	n	h/d	$(h_0, \ldots, h_{d-1})$	$(K, B, t, k, m_{\max})$	w	`len`	bitsec	pk	sk	sig
Kylin-128	128/80	2^{64}	16	62/7	$(8,9,9,9,9,9,9)$	$(4,7,128,6,95)$	17	33	127.055	32	64	6616
Kylin-256	256/128	2^{64}	32	65/8	$(8,8,8,8,8,8,8,9)$	$(9,8,66,5,187)$	16	66	255.156	64	128	26440
Kylin-512	512/256	2^{64}	64	64/8	$(8,8,8,8,8,8,8,8)$	$(14,8,256,5,415)$	43	96	511.663	128	256	84360

[2]It should be noted that the w`len` bound for calls to $\mathbf{F}$ is a worst-case bound. This is a bound on the cost for CS-WOTS$^+$ signature verification. Given that the messages are hash values which can be assumed to be close to uniformly distributed, this value will be closer to the average-case bound $(w/2)$`len` in actual measurements.

8.2 Instantiations of Hash Functions

For the NGCC submission package, hash and XOF calls follow the ICCS auxiliary interface used by the bundled implementation code. In particular, the current package uses `sm3hash`, `pseudohash`, and `pseudoXOF`, with domain-separated wrappers built on top of `pseudoXOF` in the `hash_iccs.c` and `thash_iccs.c` modules.

Recall that n and m are the security parameter and the message-digest length in bytes.

The implementation-level mapping is summarized as follows:

$$\begin{aligned}
\mathbf{H_{msg}}(\mathtt{R}, \mathtt{PK.seed}, \mathtt{PK.root}, M) &= \text{pseudoXOF}(\mathtt{tag}_{H_msg} \| \mathtt{R} \| \mathtt{PK.seed} \| \mathtt{PK.root} \| M, m),\\
\mathbf{PRF}(\mathtt{PK.seed}, \mathtt{SK.seed}, \mathtt{ADRS}) &= \text{pseudoXOF}(\mathtt{tag}_{PRF} \| \mathtt{PK.seed} \| \mathtt{ADRS} \| \mathtt{SK.seed}, n),\\
\mathbf{PRF_{msg}}(\mathtt{SK.prf}, \mathtt{OptRand}, M) &= \text{pseudoXOF}(\mathtt{tag}_{PRFmsg} \| \mathtt{SK.prf} \| \mathtt{OptRand} \| M, n),\\
\mathbf{F}(\mathtt{PK.seed}, \mathtt{ADRS}, M_1) &= \text{pseudoXOF}(\mathtt{tag}_{F} \| \mathtt{PK.seed} \| \mathtt{ADRS} \| M_1, n),\\
\mathbf{H}(\mathtt{PK.seed}, \mathtt{ADRS}, M_1 \| M_2) &= \text{pseudoXOF}(\mathtt{tag}_{H} \| \mathtt{PK.seed} \| \mathtt{ADRS} \| M_1 \| M_2, n),\\
\mathbf{T}_\ell(\mathtt{PK.seed}, \mathtt{ADRS}, M) &= \text{pseudoXOF}(\mathtt{tag}_{T} \| \mathtt{PK.seed} \| \mathtt{ADRS} \| M, n).
\end{aligned}$$

Here, `sm3hash` is the fixed-output SM3 interface and `pseudohash`/`pseudoXOF` provide the variable-length hash and XOF behavior required by the NGCC API profile. The public packaged implementation uses explicit domain-separation tags for these calls and keeps the address/seed layout aligned with the ICCS helper functions.

9 Design Rationale

The design rationale behind Kylin is to follow the original SPHINCS$^+$ construction, apply the optimized CS-WOTS$^+$ one-time signature scheme, replace the bottom few-time layer by BPORS+FP, and use a Diverse Layer Heights optimization for the hypertree. The idea behind SPHINCS$^+$ was as follows. One can build a stateless hash-based signature scheme using a massive binary certification tree and selecting a leaf at random for each message to be signed. The problem with this approach is that the tree has to be extremely high, i.e., a height of about twice the security level would be necessary. This leads to impractical signature sizes. Using a hypertree instead of a binary certification tree allows the design to trade speed for signature size. However, this is still not sufficient by itself to get practical sizes and speed.

Mathematical hard problem. Kylin is a stateless hash-based post-quantum digital signature scheme. Its security is inherited from the hardness of generic attacks against the underlying hash and XOF primitives, following the SPHINCS$^+$ proof framework. At the construction level, Kylin can be viewed as an improvement of the NIST-standardized SPHINCS+ design: it keeps the hash-based hypertree, replaces FORS by BPORS+FP, and replaces the Winternitz one-time signature layer with CS-WOTS$^+$.

Parameter selection criteria. The primary parameter-selection rule is to improve on comparable SPHINCS$^+$-style profiles while keeping the implementation and analysis close to the original construction. Under the fixed NGCC security levels, the search gives priority to smaller signatures, then faster verification, and then faster signing, while keeping the one-key signing budget at $q = 2^{64}$. The selected instances in Table 3 therefore use three main design levers: the constant-sum CS-WOTS$^+$ encoding, the BPORS+FP parameters $(K, B, t, k, m_{\max})$, and the per-layer hypertree profile $(h_0, \ldots, h_{d-1})$.

Since one-time signatures are used extensively in the SPHINCS$^+$ hypertree structure, optimizing the one-time signature size has a direct influence on the overall performance. With CS-WOTS$^+$, Kylin can retune the signature scheme to obtain smaller signatures while preserving comparable running time. BPORS+FP further reduces the bottom few-time payload by bucket-thinning child PORS keys [28] and enforcing a fixed global authentication-node budget, following the forced-pruning size-control viewpoint of [1]. Diverse Layer Heights removes the unnecessary restriction that h be divisible by d: the layers may have different heights as long as their sum is h, which lets the search use profiles such as $(8, 8, 8, 8, 8, 8, 8, 9)$ for `Kylin-256`.

Failure probability and security impact. Kylin has no decryption- or decapsulation-style correctness failure event. For honestly generated keys and honestly generated signatures, verification is deterministic and accepts except for implementation faults or violation of the underlying cryptographic

assumptions. The probability term used in the parameter search is therefore a forgery-success bound, not a correctness-failure probability.

For a signing budget of q signatures under one key pair, the concrete parameter-search exponent is derived from the BPORS+FP cover term in Eq. (9). The selected primary instances set $q = 2^{64}$ and give the bit-security exponents shown in Table 3. Thus the relevant probability term is bounded at the required NGCC levels for the three primary instances. These exponents are used to judge the impact of repeated bottom-key selection and generic forgery events on the claimed security levels.

Other considerations. The repository also maintains $q = 2^{24}$ companion profiles as additional implementations. These profiles are not the primary NGCC submission instances, but they are useful for applications with a smaller per-key signing budget and stricter signature-size preferences; see Appendix B.

We are able to prove that the encoding scheme in CS-WOTS$^+$ is size optimal among all encoding schemes over tree structures, even including those not necessarily efficiently computable. Moreover, to the best of our knowledge, no existing DAG-based scheme achieves better performance in terms of signature size, which renders our scheme the size-optimal one among all known schemes [32]. We are aware of probabilistic encoding schemes based on rejection sampling [20], and we consider those techniques to be of independent interest.

10 Security Statements and Analyses

This section gives the security claim and proof outline for the Kylin signature scheme specified in Section 7. The reduction follows the tight SPHINCS$^+$ proof framework of Hülsing and Kudinov [17], while replacing the WOTS+ checksum argument by the constant-sum antichain argument of CS-WOTS$^+$ [32], and replacing the FORS few-time layer by BPORS+FP [28, 1]. The recommended NGCC-aligned parameter families are recorded in Table 3, and the packaged ICCS hash/XOF interface is described in Section 8.2.

10.1 Security Model and Notation

Throughout this section, n denotes the Kylin security parameter in bytes and $N = 8n$ denotes the corresponding bit length. All basic hash-chain values, secret-key elements, public-key elements, and Merkle-tree nodes are n-byte strings, i.e., elements of $\{0,1\}^N$. The Winternitz parameter is w, the CS-WOTS$^+$ chain count is $\ell = \texttt{len}$, the BPORS+FP parameters are $(K, B, t, k, m_{\max})$, and the hypertree has total height h split into per-layer heights $(h_0, \ldots, h_{d-1})$ with $\sum_{i=0}^{d-1} h_i = h$.

The tweakable hash-function collection is parameterized by the public seed `PK.seed` and by 32-byte addresses `ADRS`. In this collection, $\mathbf{F}$ maps one n-byte input to one n-byte output, $\mathbf{H}$ maps two n-byte inputs to one n-byte output, and $\mathbf{T}_r$ compresses r n-byte inputs to one n-byte output. Kylin also uses the key-generation PRF $\mathbf{PRF}$, the message randomization PRF $\mathbf{PRF_{msg}}$, and the message digest function $\mathbf{H_{msg}}$.

For a primitive or scheme X, $\mathbf{InSec}^{\mathsf{sec}}(X; \tau, q)$ denotes the maximum advantage of any adversary of running time at most τ making at most q relevant oracle or target queries in the indicated security experiment. The analysis assumes the standard post-quantum model used in the SPHINCS$^+$ proof: the adversary may perform quantum computation, while the keyed signing and hashing oracles in the reduction are accessed classically.

Kylin relies on the following properties.

- $\mathbf{PRF}$ and $\mathbf{PRF_{msg}}$ are pseudorandom functions.
- $\mathbf{H_{msg}}$ satisfies interleaved target subset resilience (ITSR).
- $\mathbf{F}$, as a member of the tweakable-hash collection, satisfies single-function multi-target undetectability (SM-UD), target-collision resistance (SM-TCR), preimage resistance (SM-PRE), and decisional second-preimage resistance (SM-DSPR) for distinct tweaks.
- $\mathbf{H}$, $\mathbf{T}_K$, and $\mathbf{T}_\ell$ satisfy SM-TCR for distinct tweaks.

The distinct-tweak condition is enforced by the address scheme in Section 3.6.3. The reductions below only place targets at addresses that are fixed by a virtual BPORS+FP, XMSS, or CS-WOTS$^+$ instance; all other evaluations are simulated using the same tweakable-hash collection.

10.2 Security of CS-WOTS-TW

The one-time signature layer used in the proof is CS-WOTS-TW: the WOTS-TW scheme of [17] with the CS-WOTS$^+$ constant-sum encoding in place of the WOTS+ message-plus-checksum encoding. This subsection records the encoding property and the single-instance and multi-instance reductions.

10.2.1 The Constant-Sum Antichain Property

Let $[w] = \{0, \dots, w-1\}$, and define the product partial order on $[w]^\ell$ by

$$\mathbf{x} \preceq \mathbf{y} \quad \Longleftrightarrow \quad x_i \le y_i \text{ for every } i \in \{1, \dots, \ell\}.$$

For an integer s, define the constant-sum level

$$\mathcal{C}_s = \left\{ \mathbf{x} \in [w]^\ell : \sum_{i=1}^{\ell} x_i = s \right\}.$$

Lemma 10.1 (Constant-sum antichain)**.** *For every s, the set $\mathcal{C}_s$ is an antichain in $([w]^\ell, \preceq)$.*

Proof. Suppose $\mathbf{x}, \mathbf{y} \in \mathcal{C}_s$, $\mathbf{x} \preceq \mathbf{y}$, and $\mathbf{x} \neq \mathbf{y}$. Then at least one coordinate is strictly smaller in $\mathbf{x}$ while no coordinate is larger, so $\sum_i x_i < \sum_i y_i$, contradicting $\sum_i x_i = \sum_i y_i = s$. □

Let $D_{r,m}$ be the number of r-tuples in $\{0, \dots, w-1\}^r$ whose coordinates sum to m. The dynamic program used by the encoder is

$$D_{0,0} = 1, \qquad D_{0,m} = 0 \ (m \neq 0), \qquad D_{r,m} = \sum_{i=0}^{w-1} D_{r-1,m-i}, \tag{2}$$

with $D_{r,m} = 0$ outside the interval $0 \le m \le r(w-1)$. The Kylin parameter ℓ is selected so that, for $s^\star = \lfloor \ell(w-1)/2 \rfloor$,

$$D_{\ell,s^\star} \ge 2^N. \tag{3}$$

The encoder ranks the first 2^N elements of $\mathcal{C}_{s^\star}$ using the recurrence in Eq. (2).

Lemma 10.2 (CS-WOTS$^+$ forgery direction)**.** *Let $M \neq M'$ be two N-bit inputs to the CS-WOTS$^+$ encoder, and let $\mathbf{b} = \mathsf{cs\text{-}wots\text{-}encode}(M)$ and $\mathbf{b}' = \mathsf{cs\text{-}wots\text{-}encode}(M')$. Then there exist indices j and j' such that $b'_j < b_j$ and $b_{j'} < b'_{j'}$.*

Proof. Both encoded vectors lie in the same constant-sum level $\mathcal{C}_{s^\star}$. By Lemma 10.1, they are incomparable. If no coordinate satisfied $b'_j < b_j$, then $\mathbf{b} \preceq \mathbf{b}'$, a contradiction. The symmetric argument gives an index j' with $b_{j'} < b'_{j'}$. □

This lemma is the only place where the Kylin proof differs from the WOTS-TW part of the tight SPHINCS$^+$ proof. In WOTS+, the checksum guarantees that a changed message decreases at least one chain index. In CS-WOTS$^+$, the same "decreasing chain" needed for preimage extraction follows directly from the antichain property.

10.2.2 Single-Instance Game Proof

Theorem 10.1 (Single-instance CS-WOTS-TW security)**.** *Let $N = 8n$, let $w = \mathrm{poly}(N)$, and let CS-WOTS-TW use ℓ chains with the encoder above. If $\mathbf{PRF}$ is a PRF and $\mathbf{F}$ satisfies SM-UD, SM-TCR, and SM-PRE for distinct tweaks, then*

$$\begin{aligned}\mathbf{InSec}^{\mathsf{EU\text{-}naCMA}}(\mathsf{CS\text{-}WOTS\text{-}TW}; \tau, 1) \le{}& \mathbf{InSec}^{\mathsf{prf}}(\mathbf{PRF}; \tau, \ell) + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathbf{F}; \tau, \ell w) \\ &+ \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathbf{F}; \tau, \ell) + w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathbf{F}; \tau, \ell),\end{aligned} \tag{4}$$

up to the usual additive simulation time for at most ℓw evaluations of $\mathbf{F}$.

Proof. Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ be a one-time EU-naCMA adversary. In the real game G_1, $\mathcal{A}_1$ chooses one message M, the challenger generates one CS-WOTS-TW key pair under a context $(\mathtt{PK.seed}, \mathtt{ADRS})$, returns a signature on M, and $\mathcal{A}_2$ wins by outputting a valid signature on $M' \neq M$.

Game $\mathsf{G}_1 \to \mathsf{G}_2$: PRF replacement. In G_2, the ℓ secret chain starts $\mathbf{PRF}(\mathtt{SK.seed}, \mathtt{ADRS}_i)$ are replaced by independent uniform N-bit strings. The address scheme makes the ℓ PRF inputs distinct, hence

$$|\Pr[\mathsf{G}_1] - \Pr[\mathsf{G}_2]| \le \mathbf{InSec}^{\mathsf{prf}}(\mathbf{PRF}; \tau, \ell).$$

Game $\mathsf{G}_2 \to \mathsf{G}_3$: SM-UD hybrid. Let $\mathbf{b} = \mathsf{cs\text{-}wots\text{-}encode}(M)$. In G_3, the signature elements are generated from random hidden values one step before the revealed chain positions: for $b_i > 0$, choose $u_i \leftarrow \{0,1\}^N$ and set $\sigma_i = \mathbf{F}(\mathtt{PK.seed}, T_{i,b_i-1}, u_i)$; for $b_i = 0$, set $\sigma_i = u_i$. The public key is then completed from these signature positions exactly as in verification. A standard hybrid over the at most $w-1$ chain positions distinguishes real chain values from random values using ℓ SM-UD targets per hybrid step, and therefore

$$|\Pr[\mathsf{G}_2] - \Pr[\mathsf{G}_3]| \le w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathbf{F}; \tau, \ell).$$

Game $\mathsf{G}_3 \to \mathsf{G}_4$: excluding chain collisions. Game G_4 aborts if the forgery (M', σ') contains an index i with $b'_i < b_i$ for which

$$\mathsf{chain}_{b'_i, b_i - b'_i}(\sigma'_i, i, \mathtt{PK.seed}, \mathtt{ADRS}) = \sigma_i.$$

If this event occurs, the adversarial chain and the simulated honest chain converge below the signed position. Taking the first point at which the two paths differ but have the same image gives a valid SM-TCR solution for one of at most ℓw targeted chain calls. Thus

$$|\Pr[\mathsf{G}_3] - \Pr[\mathsf{G}_4]| \le \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathbf{F}; \tau, \ell w).$$

Bounding G_4: SM-PRE extraction. If $\mathcal{A}$ wins in G_4, then Lemma 10.2 gives an index j with $b'_j < b_j$. The abort condition tells us that the forged chain has not already reached the signed value σ_j at position b_j, while verification tells us that it does reach the same chain head. Hence the value

$$\mathsf{chain}_{b'_j, b_j - b'_j - 1}(\sigma'_j, j, \mathtt{PK.seed}, \mathtt{ADRS})$$

is a preimage of the SM-PRE target $\sigma_j = \mathbf{F}(\mathtt{PK.seed}, T_{j,b_j-1}, u_j)$. The extracted value is not the hidden preimage u_j, otherwise the paths would have met before the position excluded in G_4. Placing one SM-PRE challenge at each signed chain position yields

$$\Pr[\mathsf{G}_4] \le \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathbf{F}; \tau, \ell).$$

Combining the game transitions proves the theorem. □

10.2.3 Multi-Instance Security

In Kylin, many CS-WOTS-TW instances share the same public seed but are separated by addresses. We write D for the number of one-time instances in this multi-instance experiment, avoiding the symbol d reserved for hypertree layers.

Theorem 10.2 (Multi-instance CS-WOTS-TW security)**.** *Under the assumptions of Theorem 10.1, for D addressed CS-WOTS-TW instances sharing the same public seed,*

$$\begin{aligned}
&\mathbf{InSec}^{\mathsf{D\text{-}EU\text{-}naCMA}}(\mathsf{CS\text{-}WOTS\text{-}TW}; \tau, D) \\
&\le \mathbf{InSec}^{\mathsf{prf}}(\mathbf{PRF}; \tau, D\ell) + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathbf{F} \in \mathcal{T}_{\mathsf{H}}; \tau, D\ell w) \\
&+ \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathbf{F} \in \mathcal{T}_{\mathsf{H}}; \tau, D\ell) + w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathbf{F} \in \mathcal{T}_{\mathsf{H}}; \tau, D\ell).
\end{aligned} \tag{5}$$

Proof. Run the four games of Theorem 10.1 simultaneously over all D addressed instances. The PRF replacement uses $D\ell$ distinct inputs. The SM-UD hybrid still has at most $w-1$ chain-position hybrids, but each hybrid now contains $D\ell$ targets. The SM-TCR placement covers at most $D\ell w$ chain calls. Finally, if the adversary forges in instance r, Lemma 10.2 gives a decreasing chain coordinate in that same instance, and the SM-PRE extraction uses one of the $D\ell$ placed targets. All non-challenge evaluations are simulated through the tweakable-hash collection, and address separation ensures disjoint tweaks. □

10.3 Hypertree Security of Kylin

10.3.1 Proof-Oriented Hypertree Description

The full Kylin signature scheme is specified in Section 7. For the security proof, the relevant structure is the following. The signer first computes

$$R \leftarrow \mathbf{PRF_{msg}}(\mathtt{SK.prf}, \mathsf{opt}, M), \qquad (\mathtt{md}, \mathsf{idx_{tree}}, \mathsf{idx_{leaf}}) \leftarrow \mathbf{H_{msg}}(R, \mathtt{PK.seed}, \mathtt{PK.root}, M).$$

The digest $\mathtt{md}$ is signed by BPORS+FP. The recovered BPORS+FP public key is then signed by the bottom XMSS tree of the hypertree; each higher XMSS layer signs the root of the layer below.

Layer i has height h_i. It therefore contributes one CS-WOTS$^+$ signature and one authentication path of h_i n-byte nodes to each hypertree signature. The number of XMSS trees at layer i is

$$2^{\,h-\sum_{j=0}^{i} h_j},$$

so the top layer contains one tree. The total number of hypertree leaves is 2^h, and the total authentication-path size across all layers is hn bytes, independent of the distribution of the h_i.

10.3.2 Reduction Theorem

Theorem 10.3 (EUF-CMA security of Kylin)**.** *Let Kylin use parameters $n, w, h, d, (h_0, \ldots, h_{d-1}), \ell$ and BPORS+FP parameters $(K, B, t, k, m_{\max})$ as specified above. For any adversary making at most q_s signing queries,*

$$\begin{aligned}\mathbf{InSec}^{\mathsf{EU\text{-}CMA}}(\mathsf{Kylin}; \tau, q_s) \le\ & \mathbf{InSec}^{\mathsf{prf}}(\mathbf{PRF}; \tau, q_1) + \mathbf{InSec}^{\mathsf{prf}}(\mathbf{PRF_{msg}}; \tau, q_s) + \mathbf{InSec}^{\mathsf{itsr}}(\mathbf{H_{msg}}; \tau, q_s) \\ & + w \cdot \mathbf{InSec}^{\mathsf{sm\text{-}ud}}(\mathbf{F} \in \mathcal{T}_{\mathsf{H}}; \tau, q_2) + \mathbf{InSec}^{\mathsf{sm\text{-}pre}}(\mathbf{F} \in \mathcal{T}_{\mathsf{H}}; \tau, q_2) \\ & + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathbf{F} \in \mathcal{T}_{\mathsf{H}}; \tau, q_3 + q_7) + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathbf{H} \in \mathcal{T}_{\mathsf{H}}; \tau, q_4) \\ & + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathbf{T}_K \in \mathcal{T}_{\mathsf{H}}; \tau, q_5) + \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathbf{T}_\ell \in \mathcal{T}_{\mathsf{H}}; \tau, q_6) \\ & + 3 \cdot \mathbf{InSec}^{\mathsf{sm\text{-}tcr}}(\mathbf{F} \in \mathcal{T}_{\mathsf{H}}; \tau, q_8) + \mathbf{InSec}^{\mathsf{sm\text{-}dspr}}(\mathbf{F} \in \mathcal{T}_{\mathsf{H}}; \tau, q_8), \end{aligned} \tag{6}$$

where the target counts can be bounded by

$$q_1 < 2^{h+1}(K2^B t + \ell), \quad q_2 < 2^{h+1}\ell, \quad q_3 < 2^{h+1}\ell w, \quad q_4 < 2^{h+1}K2^B(2t+2),$$
$$q_5 < 2^h, \quad q_6 < 2^{h+1}, \quad q_7 < 2^{h+1}K2^B t, \quad q_8 < 2^h K2^B t.$$

Proof. The proof adapts the tight SPHINCS$^+$ game sequence of [17] to the CS-WOTS$^+$ and BPORS+FP layers. The hypertree, PRF, message-randomization, and tweakable-hash games are unchanged in structure; the target counts change because the bottom few-time layer now contains $K2^B$ child PORS trees per bottom key and because CS-WOTS$^+$ uses ℓ chains.

For the $\mathbf{H}$ target count, a left-filled BPORS+FP child tree with t leaves contains fewer than $2t$ hash nodes, and each outer bucket tree for one coordinate contains fewer than 2^B hash nodes. Thus one bottom BPORS+FP key contributes fewer than $K2^B(2t+2)$ relevant $\mathbf{H}$ targets, including a slack term that also covers the outer-tree nodes. The remaining XMSS binary tree nodes are bounded by the same global factor 2^{h+1} and are absorbed by the conservative q_4 bound.

Game G_0: real EUF-CMA experiment. The adversary receives a real Kylin public key and may ask for up to q_s signatures. It wins by outputting a valid signature on a message that was not queried.

Game $\mathsf{G}_0 \to \mathsf{G}_1$: replace PRF. All secret values derived from $\mathbf{PRF}(\mathtt{SK.seed}, \cdot)$ in BPORS+FP and in the CS-WOTS$^+$ instances are replaced by independent uniform values. The number of possible targeted secret values in the virtual structure is bounded by $q_1 < 2^{h+1}(K2^B t + \ell)$, giving the first PRF term in Eq. (6).

Game $\mathsf{G}_1 \to \mathsf{G}_2$: replace $\mathbf{PRF_{msg}}$. The message randomizer $\mathbf{PRF_{msg}}$ is replaced by a random function on the at most q_s signing queries. This gives the $\mathbf{InSec}^{\mathsf{prf}}(\mathbf{PRF_{msg}}; \tau, q_s)$ term.

Game $\mathsf{G}_2 \to \mathsf{G}_3$. This is the ITSR step for $\mathbf{H_{msg}}$. The game aborts if the forgery avoids introducing a fresh BPORS+FP child leaf while still corresponding to an unqueried message. This is exactly the event controlled by interleaved target subset resilience of $\mathbf{H_{msg}}$, giving $\mathbf{InSec}^{\mathsf{itsr}}(\mathbf{H_{msg}}; \tau, q_s)$.

Games $\mathsf{G}_{3.1}$–$\mathsf{G}_{3.4}$: excluding hash collisions outside WOTS. The remaining successful forgery is decomposed by the first place where the verification computation diverges from the honestly generated virtual structure.

- A second preimage in a BPORS+FP child tree, BPORS+FP outer tree, or XMSS binary tree reduces to SM-TCR of $\mathbf{H}$. The child-tree contribution is bounded by $K2^B \cdot 2t$ targets per bottom key, the outer BPORS+FP trees are covered by the extra $K2^B \cdot 2$ slack term, and the XMSS nodes are absorbed in the same conservative global bound $q_4 < 2^{h+1}K2^B(2t+2)$.
- A second preimage in the compression of the K BPORS+FP outer roots reduces to SM-TCR of $\mathbf{T}_K$ with at most $q_5 < 2^h$ targets.
- A second preimage in the compression of a CS-WOTS$^+$ public key reduces to SM-TCR of $\mathbf{T}_\ell$ with at most $q_6 < 2^{h+1}$ targets.
- A second preimage in a BPORS+FP child leaf computation outside the WOTS instances reduces to SM-TCR of $\mathbf{F}$ with at most $q_7 < 2^{h+1}K2^B t$ targets.

Game $\mathsf{G}_{3.4} \to \mathsf{G}_4$: WOTS forgery. Once the above collision events are excluded, any remaining forgery in the hypertree must contain a valid new CS-WOTS-TW signature for one of fewer than 2^{h+1} addressed one-time instances. Applying Theorem 10.2 with $D < 2^{h+1}$ gives the SM-UD, SM-PRE, and SM-TCR terms with target counts q_2 and q_3.

Final game: second BPORS+FP child leaf case. The only remaining case is the analogous "second leaf" case for BPORS+FP: the forgery contains a fresh child leaf value that verifies under the same child and outer paths but is not the expected leaf. As in the tight SPHINCS$^+$ proof, this reduces to three SM-TCR placements and one SM-DSPR term for $\mathbf{F}$, with $q_8 < 2^h K2^B t$ targets. Summing all transition losses proves Eq. (6). □

Remark 1. *The height vector $(h_0, \ldots, h_{d-1})$ affects running time and memory through the cost of generating individual XMSS trees, but not the reduction structure. The security target counts above depend on the total number of virtual leaves, WOTS instances, BPORS+FP child leaves, and authentication-path nodes, all bounded using the total height h and the BPORS+FP parameters.*

10.4 Concrete Security Against Generic Attacks

Theorem 10.3 is a reduction theorem. For parameter selection, Kylin uses a BPORS+FP cover-term proxy for random-function-like instantiations. Let q_s be the allowed number of signatures under one key. For a child PORS tree with t leaves and k opened leaves, define the fixed-child cover term after r uses by the inclusion-exclusion formula

$$p_{\mathsf{ch}}(r) = \sum_{i=0}^{k} (-1)^i \binom{k}{i} \left(\frac{\binom{t-i}{k}}{\binom{t}{k}} \right)^r . \tag{7}$$

If the bottom BPORS+FP key has already been used γ times, a fixed child bucket in one coordinate has thinned use count $R \sim \mathrm{Bin}(\gamma, 2^{-B})$. The one-coordinate averaged child cover term is therefore

$$\bar{p}_{\mathsf{ch}}(\gamma) = \mathbb{E}_{R \sim \mathrm{Bin}(\gamma, 2^{-B})} \left[p_{\mathsf{ch}}(R) \right]. \tag{8}$$

The dominant generic forgery probability used in the parameter search is

$$p_{\mathsf{gen}} = 2^{-N} + \sum_{\gamma=0}^{q_s} \left(\bar{p}_{\mathsf{ch}}(\gamma) \right)^K \binom{q_s}{\gamma} \left(1 - \frac{1}{2^h} \right)^{q_s - \gamma} 2^{-h\gamma}. \tag{9}$$

Here γ is the number of previous signatures that hit the same hypertree leaf as the forgery attempt. Conditioned on this hit count, the first factor inside the sum is the probability that all K selected BPORS+FP coordinates are covered by child secrets exposed in previous signatures, after the 2^{-B} bucket thinning in each coordinate. The leading term $2^{-N} = 2^{-8n}$ captures generic search against an n-byte primitive output. This conservative estimate does not count forced-pruning acceptance as an independent security gain; $m_{\max}$ is used for fixed-size signatures and signing-time control.

The classical generic bit-security proxy reported below is

$$\mathsf{bitsec}_{\mathsf{cl}} = -\log_2 p_{\mathsf{gen}}, \tag{10}$$

where small constant factors inside the logarithm are neglected. The profile names are submission names; the table below reports the classical exponents obtained by Eqs. (9) and (10).

Instance	q_s	n	h	d	$(K, B, t, k, m_{\max})$	w	ℓ	$\mathsf{bitsec}_{\mathsf{cl}}$
Kylin-128	2^{64}	16	62	7	$(4, 7, 128, 6, 95)$	17	33	127.055
Kylin-256	2^{64}	32	65	8	$(9, 8, 66, 5, 187)$	16	66	255.156
Kylin-512	2^{64}	64	64	8	$(14, 8, 256, 5, 415)$	43	96	511.663
Kylin-128-q24	2^{24}	16	22	1	$(2, 12, 864, 7, 90)$	6	52	127.150
Kylin-256-q24	2^{24}	32	21	1	$(3, 14, 3072, 8, 202)$	6	102	255.588
Kylin-512-q24	2^{24}	64	41	2	$(7, 15, 224, 10, 336)$	12	145	511.192

10.5 Relation to the Tight SPHINCS$^+$ Proof

The proof above inherits the tight SPHINCS$^+$ structure of [17] and changes two construction-specific parts: the WOTS encoding argument and the bottom few-time cover calculation.

1. The SPHINCS$^+$ WOTS+ checksum is replaced by Lemma 10.2. This gives the decreasing chain coordinate required for SM-PRE extraction.
2. The value of ℓ is no longer the base-w message length plus a checksum length. Instead, it is the smallest integer satisfying Eq. (3). This changes the concrete target counts q_1, q_2, q_3, q_6 but not the game sequence.
3. The SM-UD, SM-TCR, and SM-PRE hybrids for WOTS chains are otherwise the same as in WOTS-TW: they depend on chain positions and address separation, not on how the message-to-chain-index vector was encoded.
4. The diverse hypertree heights change the algorithmic loops in `ht_sign` and `ht_verify`, but the reduction uses the total height h to bound the number of virtual leaves and targets.
5. The FORS cover term is replaced by the BPORS+FP cover term in Eqs. (7) and (8). The game structure is the same: after hash-collision and WOTS-forgery events are excluded, a remaining bottom-layer forgery must either be covered by previously revealed few-time secrets or yield a second child leaf for $\mathbf{F}$.

10.6 Other Security Properties

No additional formal security claims are made in this specification. In particular, the current text does not claim dedicated side-channel resistance, perfect forward security, a quantified multi-key reduction beyond the single-key target counts above, or a separate theorem for temporary key reuse.

10.7 Security of the ICCS Hash/XOF Instantiation

The packaged NGCC implementation uses the ICCS auxiliary functions `sm3hash` and `pseudohash`, with XOF functionality provided by `pseudoXOF`. At the scheme level, $\mathbf{H}_{\mathbf{msg}}$, $\mathbf{PRF}$, $\mathbf{PRF}_{\mathbf{msg}}$, $\mathbf{F}$, $\mathbf{H}$, and $\mathbf{T}$ are realized through domain-separated wrappers over `pseudoXOF` with explicit tags and seed/address separation in the input layout.

Accordingly, the security claim in this submission is that these instantiated functions provide the pseudorandomness, target-collision resistance, preimage resistance, and related tweakable-hash properties required by Theorem 10.3 at the selected parameter sizes. This document does not make an additional independent tightness claim about the internals of the ICCS helper functions beyond that standard generic-analysis interpretation.

11 Performance Evaluation

This section provides the performance analysis and benchmark data required by the call. The current repository uses repository-local measurements for the packaged Kylin NGCC instances.

Performance analysis. At the design level, Kylin's dominant costs are hash and XOF calls. Key generation builds the top hypertree layer, signing computes BPORS+FP material and one authentication path through each hypertree layer, and verification recomputes the BPORS+FP public key and verifies the hypertree signature. CS-WOTS$^+$ reduces the number of n-byte elements in each one-time signature, BPORS+FP reduces the bottom few-time signature payload, and Diverse Layer Heights keeps the signing hash-call proxy balanced even when h is not divisible by d.

Table 4: Design-level hash-call and signature-size metrics for the primary Kylin instances.

Instance	KeyGen hash calls	Sign hash calls	Verify hash calls	Signature bytes
Kylin-128	305151	2113169	2062	6616
Kylin-256	575487	2912474	4312	26440
Kylin-512	1081855	11566745	16757	84360

Benchmark platform. The software implementation is written in ISO C and is compiled with GCC 13.3.0. The reference implementations keep their default `-O2` ISO C build with warning checks enabled. The optimized 64-bit PC implementations use per-directory builds centered on `-O3 -march=native`; `Kylin-128` additionally enables link-time optimization, omitted frame pointers, loop unrolling, and `-fno-plt`. The benchmark host is an x86-64 64-bit PC environment: AMD Ryzen 7 7800X3D 8-Core Processor, 8 physical cores and 16 hardware threads, approximately 4.192 GHz as reported by `/proc/cpuinfo`, 30 GiB RAM, Ubuntu 24.04.3 LTS on Linux 6.18.33.1-microsoft-standard-WSL2. The implementation uses the ICCS auxiliary hash/XOF interfaces `sm3hash`, `pseudohash`, and `pseudoXOF`; no external cryptographic library is required for the reported package benchmarks.

Software performance, transmission, and storage cost. Table 5 reports the automated package measurements generated by the implementation benchmark driver. Each timing entry is the median of nine measured runs after three warm-up runs on a 32-byte message. The public-key, secret-key, and signature sizes are taken from the same packaged implementation API.

Table 5: Repository-local software performance and transmission/storage cost for the packaged Kylin NGCC instances. Times are milliseconds on a 32-byte message. Sizes are bytes.

Instance	Impl.	KeyGen (ms)	Sign (ms)	Verify (ms)	Pk	Sk	Sig
Kylin-128	opt	49.246	871.313	0.568	32	64	6616
Kylin-128	ref	194.948	1746.791	1.392	32	64	6616
Kylin-256	opt	98.150	997.808	1.303	64	128	26440
Kylin-256	ref	370.071	2322.339	3.077	64	128	26440
Kylin-512	opt	325.206	21804.592	9.078	128	256	84360
Kylin-512	ref	2086.508	36862.925	33.531	128	256	84360

12 Feature Statements

This section summarizes the main characteristics of Kylin in the categories requested by the call.

Innovativeness. Kylin introduces CS-WOTS$^+$, a constant-sum, size-optimal encoding for Winternitz-type one-time signatures, replaces the bottom few-time layer with BPORS+FP, and combines both with Diverse Layer Heights in the hypertree parameter search. These changes improve the signature-size and hash-call trade-off while preserving the conservative hash-based design line.

Simplicity. The scheme stays within the well-understood family of stateless hash-based signatures. Its core ingredients are tweakable hash functions, a one-time signature layer, BPORS+FP as the few-time layer, and a hypertree, with no lattice, code, or multivariate algebraic structure.

Flexibility. The parameters $(n, w, h, d, K, B, t, k, m_{\max})$ allow the construction to trade signature size, signing time, and verification time. The primary submission fixes the three `Kylin-128`, `Kylin-256`, and `Kylin-512` profiles, while the appendix records additional $q = 2^{24}$ profiles for lower signing-budget scenarios.

Compatibility. The packaged code follows the NGCC SIG API and routes hash/XOF calls through the ICCS auxiliary functions. The public workflow remains the conventional key-generation, signing, and verification interface. Because CS-WOTS$^+$ replaces WOTS$^+$, Kylin is not byte-for-byte compatible with the original SPHINCS$^+$ formats.

Extensibility. The construction is modular: replacing the hash/XOF instantiation or retuning the parameter set does not change the top-level signature workflow. The current submission, however, only claims digital-signature functionality.

Implementation performance advantages. The repository provides both ISO C reference implementations and optimized 64-bit PC implementations for the three primary instances. As shown in Table 5, the optimized implementation improves the measured key-generation, signing, and verification times over the reference implementation on the current 64-bit PC benchmark platform.

References

[1] Mehdi Abri and Jonathan Katz. Shorter hash-based signatures using forced pruning. Cryptology ePrint Archive, Report 2025/2069, 2025.

[2] Désiré André. Mémoire sur les combinaisons régulières et leurs applications. In *Annales scientifiques de l'École Normale Supérieure*, volume 5, pages 155–198, 1876.

[3] Jean-Philippe Aumasson and Guillaume Endignoux. Improving stateless hash-based signatures. In Nigel P. Smart, editor, *Topics in Cryptology – CT-RSA 2018*, volume 10808 of *Lecture Notes in Computer Science*, pages 219–242, San Francisco, CA, USA, April 16–20, 2018. Springer, Cham, Switzerland.

[4] Hacene Belbachir and Oussama Igueroufa. Congruence properties for bi s nomial coefficients and like extended ram and kummer theorems under suitable hypothesis. *Mediterranean Journal of Mathematics*, 17(1):1–14, 2020.

[5] Daniel J. Bernstein, Daira Hopwood, Andreas Hülsing, Tanja Lange, Ruben Niederhagen, Louiza Papachristodoulou, Michael Schneider, Peter Schwabe, and Zooko Wilcox-O'Hearn. SPHINCS: Practical stateless hash-based signatures. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 368–397, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.

[6] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS$^+$ signature framework. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 2129–2146, London, UK, November 11–15, 2019. ACM Press.

[7] Daniel Bleichenbacher and Ueli M. Maurer. Directed acyclic graphs, one-way functions and digital signatures. In Yvo Desmedt, editor, *Advances in Cryptology – CRYPTO'94*, volume 839 of *Lecture Notes in Computer Science*, pages 75–82, Santa Barbara, CA, USA, August 21–25, 1994. Springer Berlin Heidelberg, Germany.

[8] Jurjen N. Bos and David Chaum. Provably unforgeable signatures. In Ernest F. Brickell, editor, *Advances in Cryptology – CRYPTO'92*, volume 740 of *Lecture Notes in Computer Science*, pages 1–14, Santa Barbara, CA, USA, August 16–20, 1993. Springer Berlin Heidelberg, Germany.

[9] David A Cooper, Daniel C Apon, Quynh H Dang, Michael S Davidson, Morris J Dworkin, Carl A Miller, et al. Recommendation for stateful hash-based signature schemes. *NIST Special Publication*, 800:208, 2020.

[10] Robert P Dilworth. A decomposition theorem for partially ordered sets. In *Classic Papers in Combinatorics*, pages 139–144. Springer, 2009.

[11] Leonhard Euler. De evolutione potestatis polynomialis cuiuscunque (1+ x+ x 2+ x 3+ x 4+ etc.) n. *Nova Acta Academiae Scientiarum Imperialis Petropolitanae*, pages 47–57, 1801.

[12] Oded Goldreich. Two remarks concerning the Goldwasser-Micali-Rivest signature scheme. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO'86*, volume 263 of *Lecture Notes in Computer Science*, pages 104–110, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany.

[13] Oded Goldreich. *Foundations of Cryptography: Basic Applications*, volume 2. Cambridge University Press, Cambridge, UK, 2004.

[14] Shay Gueron and Nicky Mouha. SPHINCS-simpira: Fast stateless hash-based signatures with post-quantum security. Cryptology ePrint Archive, Report 2017/645, 2017.

[15] Douglas Zare (https://math.stackexchange.com/users/8345/douglas zare). How to express $(1 + x + x^2 + \cdots + x^m)^n$ as a power series? Mathematics Stack Exchange, 2011. URL:https://math.stackexchange.com/q/28861 (version: 2011-11-15).

[16] Andreas Hülsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. Xmss: extended merkle signature scheme. In *RFC 8391*. IRTF, 2018.

[17] Andreas Hülsing and Mikhail A. Kudinov. Recovering the tight security proof of SPHINCS$^+$. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part IV*, volume 13794 of *Lecture Notes in Computer Science*, pages 3–33, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland.

[18] Andreas Hülsing, Mikhail A. Kudinov, Eyal Ronen, and Eylon Yogev. SPHINCS+C: Compressing SPHINCS+ with (almost) no cost. In *2023 IEEE Symposium on Security and Privacy*, pages 1435–1453, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press.

[19] Dmitry Khovratovich, Mikhail A. Kudinov, and Benedikt Wagner. At the top of the hypercube - better size-time tradeoffs for hash-based signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part VI*, volume 16005 of *Lecture Notes in Computer Science*, pages 93–123, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.

[20] Mikhail Kudinov, Andreas Hülsing, Eyal Ronen, and Eylon Yogev. Sphincs+c: Compressing sphincs+ with (almost) no cost. Cryptology ePrint Archive, Paper 2022/778, 2022. `https://eprint.iacr.org/2022/778`.

[21] Leslie Lamport. Constructing digital signatures from a one-way function. Technical Report SRI-CSL-98, SRI International Computer Science Laboratory, October 1979.

[22] David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-micali hash-based signatures. In *RFC 8554*. IRTF, 2019.

[23] Ralph C. Merkle. A certified digital signature. In Gilles Brassard, editor, *Advances in Cryptology – CRYPTO'89*, volume 435 of *Lecture Notes in Computer Science*, pages 218–238, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA.

[24] National Institute of Standards and Technology. Stateless hash-based digital signature standard (fips 205). Technical report, U.S. Department of Commerce, 2024.

[25] National Institute of Standards and Technology. Recommendation for stateful hash-based signature schemes (initial public draft). Technical report, U.S. Department of Commerce, April 2026. Published April 13, 2026.

[26] Lucas Pandolfo Perin, Gustavo Zambonin, Ricardo Custódio, Lucia Moura, and Daniel Panario. Improved constant-sum encodings for hash-based signatures. *Journal of Cryptographic Engineering*, pages 1–23, 2021.

[27] Zhen Qin and Siwei Sun. Extending the SPHINCS+ framework: Varying the tree heights and chain lengths. Cryptology ePrint Archive, Report 2025/2236, 2025.

[28] Chongxu Ren, Kaiyi Zhang, and Hongbo Yu. Shorter hash-based signatures via bucket thinning. Cryptology ePrint Archive, Paper 2026/1328, 2026. `https://eprint.iacr.org/2026/1328`.

[29] Emanuel Sperner. Ein satz über untermengen einer endlichen menge. *Mathematische Zeitschrift*, 27(1):544–548, 1928.

[30] Serge Vaudenay. One-time identification with low memory. In *Eurocode'92*, pages 217–228. Springer, 1993.

[31] S Ole Warnaar. The andrews–gordon identities and q-multinomial coefficients. *Communications in mathematical physics*, 184(1):203–232, 1997.

[32] Kaiyi Zhang, Hongrui Cui, and Yu Yu. Revisiting the constant-sum winternitz one-time signature with applications to sphincs+ and xmss. In *Advances in Cryptology–CRYPTO 2023: 43th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–24, 2023, Proceedings, to appear*, pages xxx–xxx. Springer, 2023.

A Parameter Evaluation Sage Script

The script below evaluates parameter candidates with per-layer hypertree heights $(h_0, \ldots, h_{d-1})$ satisfying $\sum_{i=0}^{d-1} h_i = h$. It uses h_0 for the message-digest index split and uses $\sum_{i=0}^{d-1} 2^{h_i}$ in the HT hash-call proxy. The currently frozen rows include the packaged $q = 2^{64}$ tuples `Kylin-128/256/512` and the companion $q = 2^{24}$ tuples `Kylin-128-q24/256-q24/512-q24`.

```
#!/usr/bin/env python3
"""Reference checker for the frozen Kylin BPORS+FP parameter profiles.

This appendix script mirrors the current repository source of truth,
``paper/data/para_kylin_final_6profiles.csv``.  It verifies the six frozen
primary and companion profiles by recomputing signature size, the conservative
BPORS+FP cover exponent, and the recorded sign/verify hash-call proxies.
"""

from __future__ import annotations

import csv
import math
from math import comb
from pathlib import Path

import mpmath as mp

mp.mp.dps = 120

ROOT = Path(__file__).resolve().parents[2]
CSV = ROOT / "paper/data/para_kylin_final_6profiles.csv"
CTR_BYTES = 8
BPORS_SAMPLER_BUF_U32 = 512 // 4
SECURITY_TOLERANCE = mp.mpf("1e-12")

EXPECTED = {
    "Kylin-128": (2**64, 16, 62, 7, (8, 9, 9, 9, 9, 9, 9), 4, 7, 128, 6, 95, 17, 33, 6616),
    "Kylin-256": (2**64, 32, 65, 8, (8, 8, 8, 8, 8, 8, 8, 9), 9, 8, 66, 5, 187, 16, 66, 26440),
    "Kylin-512": (2**64, 64, 64, 8, (8, 8, 8, 8, 8, 8, 8, 8), 14, 8, 256, 5, 415, 43, 96, 84360),
    "Kylin-128-q24": (2**24, 16, 22, 1, (22,), 2, 12, 864, 7, 90, 6, 52, 2872),
    "Kylin-256-q24": (2**24, 32, 21, 1, (21,), 3, 14, 3072, 8, 202, 6, 102, 11208),
    "Kylin-512-q24": (2**24, 64, 41, 2, (20, 21), 7, 15, 224, 10, 336, 12, 145, 47240),
}


def parse_q(text: str) -> int:
    base, exp = text.split("^")
    if base != "2":
        raise ValueError(f"unsupported q label: {text}")
    return 1 << int(exp)


def parse_h_vec(text: str) -> tuple[int, ...]:
    return tuple(int(x) for x in text.strip("[]").split(",") if x)


def ceil_log2(x: int) -> int:
    return (x - 1).bit_length()


def child_cover_terms(t: int, k: int, B: int) -> tuple[list[mp.mpf], list[mp.mpf]]:
    p_bucket = mp.mpf(2) ** (-B)
    denom = mp.mpf(comb(t, k))
    coeffs: list[mp.mpf] = []
    bases: list[mp.mpf] = []
    for i in range(k + 1):
```

```
        ratio = mp.mpf(comb(t - i, k)) / denom if t - i >= k else mp.mpf(0)
        coeffs.append(mp.mpf((-1 if i % 2 else 1) * comb(k, i)))
        bases.append(1 - p_bucket + p_bucket * ratio)
    return coeffs, bases


def bpors_bitsec(q_sigs: int, n: int, h: int, K: int, B: int, t: int, k: int) -> mp.mpf:
    p_hit = mp.mpf(2) ** (-h)
    q = mp.mpf(q_sigs)
    prob = (1 - p_hit) ** q
    mass = mp.mpf(0)
    total = mp.mpf(2) ** (-(8 * n))
    coeffs, bases = child_cover_terms(t, k, B)
    powers = [mp.mpf(1) for _ in bases]

    for gamma in range(2000):
        p_child = mp.fsum(c * x for c, x in zip(coeffs, powers))
        if p_child < 0 and abs(p_child) < mp.mpf("1e-100"):
            p_child = mp.mpf(0)
        total += prob * (p_child**K)
        mass += prob
        if gamma > 500 and 1 - mass < mp.mpf("1e-90"):
            break
        next_gamma = gamma + 1
        prob *= ((q - gamma) / next_gamma) * (p_hit / (1 - p_hit))
        for i, base in enumerate(bases):
            powers[i] *= base

    return -mp.log(total, 2)


def signature_bytes(row: dict[str, str]) -> int:
    n = int(row["n"])
    h = int(row["h"])
    d = int(row["d"])
    K = int(row["bpors_K"])
    k = int(row["bpors_k"])
    mmax = int(row["bpors_mmax"])
    length = int(row["len"])
    return n + CTR_BYTES + (K * k + mmax) * n + (h + d * length) * n


def bounded_draw_factor(bound: int) -> float:
    range32 = 1 << 32
    limit = range32 - (range32 % bound)
    return range32 / limit


def expected_distinct_draws(t: int, k: int) -> float:
    return sum(t / (t - i) for i in range(k))


def sampler_blocks(words: float) -> int:
    return max(1, math.ceil(words / BPORS_SAMPLER_BUF_U32))


def bpors_index_sampler_blocks(K: int, t: int, k: int) -> int:
    words = K * expected_distinct_draws(t, k) * bounded_draw_factor(t)
    return sampler_blocks(words)


def bpors_bucket_sampler_blocks(K: int) -> int:
    return sampler_blocks(K)


def bpors_sign_base(K: int, B: int, t: int) -> int:
    return K * (1 << B) * (3 * t - 1) + K * ((1 << B) - 1)


def sign_ht_hashcalls(h_vec: tuple[int, ...], w: int, length: int) -> int:
    return sum((1 << h_i) * (length * w + 1) for h_i in h_vec)


def verify_chain_steps(length: int, w: int) -> int:
    total = length * (w - 1)
    return total - (total // 2)


def verify_ht_hashcalls(h: int, d: int, w: int, length: int) -> int:
    return d * (verify_chain_steps(length, w) + 1) + h


def bpors_sign_hashcalls(row: dict[str, str]) -> int:
    K = int(row["bpors_K"])
    B = int(row["bpors_B"])
    t = int(row["bpors_t"])
    k = int(row["bpors_k"])
```

```
    tau = float(row["tau"])
    h_vec = parse_h_vec(row["h_profile"])
    ht_sign = sign_ht_hashcalls(h_vec, int(row["w"]), int(row["len"]))
    trial_cost = bpors_index_sampler_blocks(K, t, k)
    success_cost = bpors_bucket_sampler_blocks(K)
    return ht_sign + bpors_sign_base(K, B, t) + success_cost + math.ceil(trial_cost / tau)


def bpors_verify_hashcalls(row: dict[str, str]) -> int:
    K = int(row["bpors_K"])
    B = int(row["bpors_B"])
    t = int(row["bpors_t"])
    k = int(row["bpors_k"])
    h = int(row["h"])
    d = int(row["d"])
    ht_verify = verify_ht_hashcalls(h, d, int(row["w"]), int(row["len"]))
    sampler_cost = bpors_index_sampler_blocks(K, t, k) + bpors_bucket_sampler_blocks(K)
    return ht_verify + 2 * K * k + int(row["bpors_mmax"]) + sampler_cost


def verify_row(row: dict[str, str]) -> None:
    name = row["name"]
    h_vec = parse_h_vec(row["h_profile"])
    got = (
        parse_q(row["q"]),
        int(row["n"]),
        int(row["h"]),
        int(row["d"]),
        h_vec,
        int(row["bpors_K"]),
        int(row["bpors_B"]),
        int(row["bpors_t"]),
        int(row["bpors_k"]),
        int(row["bpors_mmax"]),
        int(row["w"]),
        int(row["len"]),
        int(row["sig"]),
    )
    if got != EXPECTED[name]:
        raise AssertionError(f"parameter drift for {name}: {got}")
    if sum(h_vec) != int(row["h"]) or len(h_vec) != int(row["d"]):
        raise AssertionError(f"invalid layer-height profile for {name}")
    if int(row["bpors_child_height"]) != ceil_log2(int(row["bpors_t"])):
        raise AssertionError(f"child-height mismatch for {name}")
    if signature_bytes(row) != int(row["sig"]):
        raise AssertionError(f"signature-size mismatch for {name}")
    if bpors_sign_hashcalls(row) != int(row["sign_hashcalls"]):
        raise AssertionError(f"sign hash-call mismatch for {name}")
    if bpors_verify_hashcalls(row) != int(row["verify_hashcalls"]):
        raise AssertionError(f"verify hash-call mismatch for {name}")

    bitsec = bpors_bitsec(
        parse_q(row["q"]),
        int(row["n"]),
        int(row["h"]),
        int(row["bpors_K"]),
        int(row["bpors_B"]),
        int(row["bpors_t"]),
        int(row["bpors_k"]),
    )
    if bitsec + SECURITY_TOLERANCE < mp.mpf(row["target_bits"]):
        raise AssertionError(f"BPORS cover estimate below target for {name}")


def main() -> None:
    seen: set[str] = set()
    with CSV.open(newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            verify_row(row)
            seen.add(row["name"])
            print(
                f"{row['name']}: q={row['q']}, n={row['n']}, h={row['h']}, d={row['d']}, "
                f"h_vec={row['h_profile']}, BPORS=({row['bpors_K']},{row['bpors_B']},"
                f"{row['bpors_t']},{row['bpors_k']},{row['bpors_mmax']}), "
                f"w={row['w']}, len={row['len']}, bitsec={row['bitsec']}, sig={row['sig']}, "
                f"sign={row['sign_hashcalls']}, verify={row['verify_hashcalls']}"
            )
    if seen != set(EXPECTED):
        raise AssertionError(f"profile set mismatch: {sorted(seen)}")


if __name__ == "__main__":
    main()
```

B $q = 2^{24}$ Companion Parameter Profiles

This appendix summarizes the frozen companion $q = 2^{24}$ profiles that are kept as additional implementations. The motivation is practical: some application settings remain dissatisfied with large signature sizes and verification-time behavior under very large per-key signing budgets. In addition, NIST's April 13, 2026 initial public draft SP 800-230 highlights stateful hash-based signatures with explicit per-key usage bounds, which further motivates keeping explicit companion rows for limited-signature operating points (including 2^{24}-style limits) [25].

In this repository, the $q = 2^{24}$ rows are selected from the same constrained search workflow and then frozen as additional companions to the packaged NGCC $q = 2^{64}$ families.

Table 6: Frozen companion $q = 2^{24}$ Kylin parameter profiles (additional implementations).

Family	NGCC lvl	n	h/d	$(h_0, \ldots, h_{d-1})$	$(K, B, t, k, m_{\max})$	w	l	bitsec	pk	sk	sig
Kylin-128-q24	128/80	16	22/1	(22)	$(2, 12, 864, 7, 90)$	6	52	127.150	32	64	2872
Kylin-256-q24	256/128	32	21/1	(21)	$(3, 14, 3072, 8, 202)$	6	102	255.588	64	128	11208
Kylin-512-q24	512/256	64	41/2	(20, 21)	$(7, 15, 224, 10, 336)$	12	145	511.192	128	256	47240

The optimized implementations apply macro-unrolled SM3 compression, SSE2/AVX2 hardware dispatch for message-word loading, a malloc-free `pseudoXOF` fast path for byte-aligned input, and 4-way SSE2 WOTS chain acceleration adapted for $n = 16/32/64$. Both the sign and verify paths batch 4 WOTS chains per SIMD operation.